Universidad Nacional de Asunción
Facultad Politécnica

Ingeniería en Informática



Verificación de consenso e inclusión de transacciones en Ethereum con conocimiento cero aplicado a la transferencia de activos digitales

Javier Rafael López Cáceres
Alejandro Fabián Marín Insfran

San Lorenzo - Paraguay
Septiembre - 2026

Universidad Nacional de Asunción
Facultad Politécnica

Ingeniería en Informática



Verificación de consenso e inclusión de transacciones en Ethereum con conocimiento cero aplicado a la transferencia de activos digitales

Javier Rafael López Cáceres
Alejandro Fabián Marín Insfran

Asesor:
PhD Marcos Villagra

Proyecto de Trabajo de Grado presentado en conformidad a los requisitos para obtener el grado de Ingeniería en Informática

San Lorenzo - Paraguay
Septiembre - 2026

Dedico a mi familia,
y a mis amigos hghh.

AGRADECIMIENTOS

Verificación de consenso e inclusión de transacciones en Ethereum con conocimiento cero aplicado a la transferencia de activos digitales

Autor: Javier López - Alejandro Marín

Asesor: PhD Marcos Villagra

RESUMEN

Los puentes entre cadenas o popularmente conocidos como *cross-chain bridges*, permiten transferir activos entre distintas *blockchains* y constituyen una pieza central de la interoperabilidad. La mayoría se apoya en el patrón *lock-and-mint* operado por un *relayer* de confianza, un actor que tras observar el bloqueo de fondos en la cadena de origen acuña el equivalente en la cadena de destino. Como toda la seguridad recae en ese operador, que podría acuñar de forma fraudulenta, los puentes se han vuelto uno de los blancos más explotados del ecosistema. En este trabajo se diseña e implementa un puente de confianza reducida (*trust-minimized*) que sustituye esa confianza por verificación criptográfica. El contrato de destino solo acuña cuando valida *on-chain* una prueba de conocimiento cero (*zero-knowledge*) que demuestra tanto la inclusión real de la transacción de bloqueo en un bloque de la cadena de origen a través de una prueba de inclusión en el árbol de transacciones, como la coincidencia del destinatario y el monto de la transferencia con esa transacción, de modo que no puedan adulterarse. La lógica de verificación se desarrolló en Noir y, de forma equivalente, en Rust para la máquina virtual de conocimiento cero SP1, con verificadores *on-chain* en Solidity, y ambas implementaciones se comparan empíricamente en rendimiento, costo de verificación y viabilidad práctica. Un *relayer off-chain* construye las pruebas de consenso y de inclusión de cada operación y las remite al contrato de destino, donde deja de ser un acuñador de confianza y pasa a ser un probador sin permisos, ya que cualquiera con pruebas válidas puede liberar los fondos y nadie puede acuñar sin ellas. De manera complementaria se diseña la validación de la firma del comité de sincronización (*sync committee*) de Ethereum como raíz de confianza sobre la finalidad del bloque. La solución se validó sobre redes de prueba reales, con Ethereum Sepolia como cadena de origen y Optimism Sepolia como destino. Adicionalmente incluye una interfaz web y un modo comparativo que evidencia de forma experimental cómo el esquema clásico admite la acuñación fraudulenta mientras que el esquema con verificación de conocimiento cero la rechaza.

Palabras Clave: puentes entre cadenas (*cross-chain bridges*); pruebas de conocimiento cero (*zk-SNARKs*); Noir; UltraHonk; SP1; verificación *on-chain*; interoperabilidad *blockchain*; prueba de inclusión (árbol de Merkle-Patricia); confianza reducida (*trust-minimization*); Ethereum.

Title

Author: full name

Advisor: full name

ABSTRACT

This is the abstract

Keywords:

ÍNDICE

Página

1. INTRODUCCIÓN **1**

1.1. Contexto y antecedentes . . . 1
1.2. Planteamiento del problema . . . 2
1.3. Justificación . . . 2
1.4. Objetivos . . . 3
1.4.1. Objetivo general . . . 3
1.4.2. Objetivos específicos . . . 3
1.5. Hipótesis . . . 4
1.6. Delimitación . . . 4

2. BLOCKCHAIN **5**

2.1. Arquitectura de blockchain . . . 5
2.1.1. Estructura de bloques y encadenamiento por hash . . . 5
2.1.2. Árboles de Merkle y pruebas de inclusión . . . 6
2.1.3. Mecanismos de consenso . . . 6
2.1.4. Roles de nodos y topología de red . . . 7
2.2. Ethereum (Estructura de bloques) . . . 8
2.3. Aztec (Estructura de bloques) . . . 8
2.4. Puentes . . . 8

3. CONOCIMIENTO CERO **9**

3.1. Fundamentos . . . 9
3.1.1. Concepto de conocimiento cero . . . 9
3.1.2. Propiedades fundamentales . . . 11
3.1.3. Pruebas interactivas y no interactivas . . . 12
3.1.4. De una computación a una prueba . . . 13
3.1.5. SNARKs y STARKs . . . 16
3.2. Circom . . . 18
3.2.1. Origen y propósito . . . 18
3.2.2. Construcción de circuitos . . . 19
3.2.3. Compilación, cálculo del testigo y generación de la prueba . . . 22
3.2.4. Limitaciones y consideraciones de seguridad . . . 24
3.3. Noir . . . 26
3.3.1. Origen y propósito . . . 26
3.3.2. Descripción de computaciones y generación de restricciones . . . 27
3.3.3. Compilación, cálculo del testigo y generación de la prueba . . . 29
3.3.4. Limitaciones y consideraciones de seguridad . . . 32
3.4. SP1 . . . 34
3.4.1. Origen y propósito . . . 35

3.4.2. Programas, entradas y valores públicos 36
3.4.3. Compilación, ejecución, generación y verificación de la prueba 39
3.4.4. Limitaciones y consideraciones de seguridad 41
3.5. Otros (Risc0, Succinct, Aligned, Cairo, Halo2, Plonky3) 43

4. ZKBRIDGE **44**
4.1. Implementación de un puente Lock-and-Mint 44
4.2. zkBridge de Berkeley RDI 44
4.3. Diseño e Implementación en Noir 44
4.4. Benchmarks 44

5. CONCLUSIÓN **45**

REFERENCIAS **46**

APÉNDICE **53**
A.1. A 53

LISTA DE FIGURAS

Página

3.1. Etapas de generación y verificación de una prueba. 14
3.2. Circuito Circom que expresa la relación $w^3 + w + 5 = y$. 20
3.3. Proceso de compilación, cálculo del testigo, generación y verificación de una prueba a partir de un circuito Circom. 22
3.4. Plantilla `IsZero` de circomlib, que combina un cálculo auxiliar con restricciones explícitas. 24
3.5. Programa Noir que expresa la relación $w^3 + w + 5 = y$. 28
3.6. Proceso de compilación, cálculo del testigo, generación y verificación de una prueba a partir de un programa Noir con el backend Barretenberg. 30
3.7. Cálculo no restringido de un inverso en Noir, seguido de la restricción que lo valida. 32
3.8. Programa invitado de SP1 que calcula $y = w^3 + w + 5$ con aritmética entera comprobada. 37
3.9. Programa anfitrión que prepara la entrada, genera y verifica una prueba comprimida y lee el valor público. 38
3.10. Proceso de compilación, ejecución, generación y verificación de una prueba con SP1. 39

LISTA DE TABLAS

Página

3.1. Diferencias conceptuales entre SNARKs y STARKs. 17

LISTA DE SÍMBOLOS

H	Función de hash criptográfica.
$\|\|$	Concatenación de cadenas de bits.
$\mathcal{P}$	Probador (*prover*) de un sistema de prueba.
$\mathcal{P}^*$	Probador potencialmente deshonesto.
$\mathcal{V}$	Verificador (*verifier*) de un sistema de prueba.
x	Enunciado (*statement*); entrada pública.
w	Testigo (*witness*); entrada privada del probador.
π	Prueba (*proof*) generada por el probador.
$\mathcal{R}$	Relación que define los pares válidos de enunciado y testigo.
$\mathbb{F}_p$	Cuerpo finito de p elementos, con p primo.
A, B, C	Combinaciones lineales de señales en una restricción cuadrática $A \cdot B + C = 0$ de Circom.

CAPÍTULO 1
INTRODUCCIÓN

1.1. Contexto y antecedentes

Ethereum se ha consolidado como la plataforma de referencia para el despliegue de contratos inteligentes y constituye la base sobre la que se sostiene una porción significativa del ecosistema de finanzas descentralizadas [1]. Este crecimiento no se produjo de forma aislada, sino que estuvo acompañado por la proliferación de redes de *Layer 2* y cadenas alternativas que replican o extienden su modelo de ejecución, cada una optimizada según distintos criterios de costo, rendimiento o descentralización. La fragmentación resultante de este ecosistema plantea la necesidad de que los activos digitales y los datos puedan trasladarse entre las distintas cadenas que lo componen, requisito que se conoce como interoperabilidad. Los puentes entre cadenas, o *cross-chain bridges*, constituyen la infraestructura que hace posible dicho traslado, de modo que la seguridad de estos mecanismos determina, en gran medida, la seguridad de los fondos que a través de ellos se transfieren.

Paralelamente al desarrollo de estas redes, las pruebas de conocimiento cero (*Zero-Knowledge Proofs*) evolucionaron desde ser un objeto de estudio predominantemente teórico hasta convertirse en una herramienta de ingeniería aplicable a problemas concretos. Esta tecnología permite comprimir la verificación de un cómputo arbitrariamente extenso en una prueba sucinta, cuya validación puede realizarse en una fracción del tiempo y del costo que exigiría reejecutar dicho cómputo. Aplicada al contexto de Ethereum, esta propiedad habilita la verificación liviana, desde otra cadena, tanto del consenso alcanzado por los validadores —a través de la firma emitida por el comité de sincronización que certifica la finalidad de un bloque— como de la inclusión efectiva de una transacción particular dentro de ese bloque, lo cual se logra mediante una prueba sobre su árbol de Merkle-Patricia. La combinación de ambas verificaciones, comprimidas en una única prueba de conocimiento cero, constituye el fundamento técnico sobre el que resulta

posible construir un puente entre cadenas cuya seguridad no dependa de la honestidad de un intermediario, sino de una demostración matemáticamente verificable. En el presente trabajo, este fundamento se materializa mediante el diseño e implementación de un puente del tipo *lock-and-mint*, en el cual el bloqueo de un activo en la cadena de origen habilita su acuñación equivalente en la cadena de destino únicamente cuando dicha prueba ha sido validada *on-chain*.

1.2. Planteamiento del problema

La verificación directa del consenso de Ethereum y de la inclusión de una transacción, sin comprimir dicho cómputo mediante una prueba de conocimiento cero, resulta impracticable cuando se pretende realizar fuera de la propia cadena. Reproducir la agregación de las firmas del comité de sincronización y recorrer un árbol de Merkle-Patricia dentro de un contrato inteligente desplegado en otra cadena implica un volumen considerable de operaciones criptográficas, fundamentalmente sobre curvas elípticas, cuyo costo en gas resulta, en la práctica, prohibitivo. Esta limitación conduce a que los sistemas de interoperabilidad existentes adopten un compromiso distinto, en el cual la verificación criptográfica del estado de la cadena de origen es sustituida por la delegación de dicha verificación en un comité de validadores externos sobre el cual se deposita confianza. Este diseño traslada el problema de seguridad del propio protocolo hacia el comportamiento honesto de un conjunto reducido de actores, y esta delegación de confianza ha constituido, de manera sistemática, el punto de falla explotado en los incidentes de seguridad más costosos sufridos por puentes entre cadenas hasta la fecha [2]. Persiste, en consecuencia, una tensión no resuelta entre dos objetivos igualmente necesarios: por un lado, alcanzar garantías criptográficas equivalentes a las del consenso de la cadena de origen y, por otro, mantener un costo de verificación *on-chain*, tanto en gas como en tiempo de generación de la prueba, que resulte viable para un sistema de uso real.

1.3. Justificación

El ecosistema de herramientas orientadas al desarrollo de circuitos de conocimiento cero capaces de abordar este volumen de cómputo criptográfico se encuentra, a su vez, en un proceso activo de maduración, y ofrece actualmente dos paradigmas de diseño claramente diferenciados. El primero está representado por los lenguajes de dominio específico, orientados exclusivamente a la escritura de circuitos, entre los cuales se encuentra Noir [3], que expone al desarrollador primitivas y un modelo de programación concebidos desde su diseño para la generación eficiente de pruebas. El segundo paradigma corresponde a las máquinas virtuales de conocimiento cero de propósito general, cuyo exponente más reciente es SP1 [4], las cuales permiten probar la ejecución de programas escritos en un lenguaje de propósito general, en este caso Rust, compilados a un conjunto de instrucciones que la máquina virtual ejecuta y prueba sin que el desarrollador deba razonar directamente en términos de restricciones aritméticas. Si bien ambos paradigmas

persiguen resolver el mismo tipo de problema, lo hacen mediante compromisos de diseño distintos entre la expresividad del lenguaje, la facilidad de desarrollo y depuración, y el rendimiento final del circuito resultante. Dado que ambas herramientas son de adopción reciente y se encuentran en desarrollo activo, no existe todavía evidencia empírica consolidada que permita fundamentar la elección de uno u otro paradigma para la implementación de un caso de uso concreto y computacionalmente exigente, como lo es la verificación de consenso e inclusión de transacciones en Ethereum. El presente trabajo se propone generar dicha evidencia mediante la implementación de un mismo problema en ambas tecnologías, bajo condiciones comparables, y la medición sistemática de su comportamiento. Ambas implementaciones se integran, además, dentro del mismo puente *lock-and-mint*, de manera que la comparación entre Noir y SP1 no se limita a un ejercicio aislado de generación de pruebas, sino que se valida en el contexto de una transferencia real de activos digitales entre cadenas.

1.4. Objetivos

1.4.1. Objetivo general

Diseñar, implementar y comparar empíricamente la verificación de consenso de Ethereum y de inclusión de transacciones mediante Noir y SP1, integrando ambas implementaciones en un puente *lock-and-mint* de confianza reducida, con el propósito de determinar cuál de ambas tecnologías ofrece un mejor rendimiento, un menor costo de verificación y una mayor viabilidad práctica para la transferencia segura de activos digitales entre cadenas.

1.4.2. Objetivos específicos

- Investigar los mecanismos de prueba y las arquitecturas de circuitos que subyacen a Noir, en tanto lenguaje de dominio específico, y a SP1, en tanto máquina virtual de conocimiento cero de propósito general basada en Rust.
- Implementar, en ambas tecnologías, la lógica de verificación de encabezados de bloque de Ethereum, de la firma del comité de sincronización y de las pruebas de inclusión de transacciones.
- Medir y comparar las métricas de rendimiento relevantes entre ambas implementaciones, entre las que se incluyen el tiempo de generación de la prueba, el uso de recursos computacionales y de memoria, el costo de verificación *on-chain* en gas, y la complejidad relativa del proceso de desarrollo.

1.5. Hipótesis

El presente trabajo se orienta por la siguiente pregunta de investigación: ¿presenta el paradigma de máquina virtual de conocimiento cero de propósito general, representado por SP1, ventajas significativas en términos de flexibilidad de desarrollo y de tiempos de ejecución frente a un lenguaje de conocimiento cero de dominio específico como Noir, al implementar la verificación de consenso e inclusión de transacciones en Ethereum?

1.6. Delimitación

El presente trabajo se circunscribe a la arquitectura de consenso de Ethereum posterior a *The Merge*, particularmente al mecanismo del comité de sincronización empleado por los clientes ligeros para certificar la finalidad de un bloque, así como a las pruebas de inclusión sobre su árbol de Merkle-Patricia. Las implementaciones desarrolladas y las comparativas de rendimiento realizadas se validan sobre redes de prueba, empleando Ethereum Sepolia como cadena de origen y Optimism Sepolia como cadena de destino, bajo una infraestructura de cómputo estandarizada que permite que las mediciones obtenidas para Noir y para SP1 resulten comparables entre sí. Quedan fuera del alcance de este trabajo tanto la implementación sobre redes principales como la verificación de mecanismos de consenso correspondientes a cadenas distintas de Ethereum.

CAPÍTULO 2

BLOCKCHAIN

2.1. Arquitectura de blockchain

Una *blockchain* es un registro distribuido (*distributed ledger*) de transacciones que se mantiene de forma replicada entre los nodos de una red *peer-to-peer*, sin depender de una autoridad central que valide o custodie su contenido. Su propiedad fundamental es que, una vez que un dato se incorpora al registro, modificarlo o eliminarlo requiere rehacer todo el trabajo criptográfico posterior a ese punto, lo cual se vuelve computacionalmente inviable a medida que la cadena crece. Esta sección describe los cuatro componentes que en conjunto sostienen esa propiedad: la estructura de bloques encadenados por hash, los árboles de Merkle que resumen su contenido, los mecanismos de consenso que deciden qué bloque es válido, y los roles que los distintos tipos de nodos cumplen dentro de la red. Estos conceptos son la base sobre la que se apoyan tanto el diseño de Ethereum (Sección 2.2) como el esquema de verificación de conocimiento cero desarrollado en este trabajo.

2.1.1. Estructura de bloques y encadenamiento por hash

Definición 2.1. Un **bloque** es una estructura de datos compuesta por un *encabezado* (*header*) y un cuerpo. El encabezado contiene, como mínimo, el hash criptográfico del encabezado del bloque anterior, una marca de tiempo, un valor que resume el conjunto de transacciones del bloque (la raíz de un árbol de Merkle, ver Definición 2.2), y metadatos propios del mecanismo de consenso utilizado (por ejemplo, un *nonce* en *Proof-of-Work*). El cuerpo contiene la lista de transacciones incluidas en el bloque.

La **blockchain** propiamente dicha es la secuencia de bloques $B_0, B_1, \ldots, B_n$ donde B_0 es el *bloque génesis* y cada bloque B_i, con $i > 0$, referencia a su predecesor incluyendo $H(B_{i-1})$

en su propio encabezado, siendo H una función de hash criptográfica (típicamente SHA-256 o Keccak-256). Esta construcción encadenada es lo que le da su nombre a la estructura y es la responsable directa de su inmutabilidad: como el hash de un bloque depende del hash del bloque anterior, alterar el contenido de B_i cambia su hash $H(B_i)$, lo que invalida la referencia almacenada en B_{i+1}, y por extensión la de todos los bloques posteriores hasta la cabeza de la cadena. Un atacante que quisiera modificar una transacción confirmada tendría entonces que recalcular el trabajo de consenso de todos los bloques desde ese punto en adelante, y hacerlo más rápido que el resto de la red honesta le suma nuevos bloques a la cadena legítima.

2.1.2. Árboles de Merkle y pruebas de inclusión

Incluir el hash de cada transacción individual en el encabezado del bloque sería costoso en espacio y haría que verificar la pertenencia de una transacción específica implicara descargar el bloque completo. Los árboles de Merkle [5] resuelven ambos problemas.

Definición 2.2. Un **árbol de Merkle** es un árbol binario en el que cada hoja contiene el hash de un elemento de datos (en este contexto, una transacción) y cada nodo interno contiene el hash de la concatenación de sus dos hijos: $h_{padre} = H(h_{izq} \| h_{der})$. El hash almacenado en la raíz, llamado **raíz de Merkle**, es un compromiso criptográfico sobre el conjunto completo de hojas: cambiar cualquier transacción altera su hash hoja y, por propagación, la raíz.

Esta estructura permite generar una **prueba de inclusión** (*Merkle proof*) de que una transacción t forma parte de un bloque sin necesitar el bloque completo: basta con entregar t junto con los hashes hermanos de cada nivel del camino desde su hoja hasta la raíz. Quien verifica recalcula los hashes de ese camino y comprueba que el resultado coincide con la raíz de Merkle publicada en el encabezado del bloque. Para un árbol con n hojas, la prueba tiene tamaño $O(\log n)$, en contraste con el $O(n)$ que implicaría transmitir todas las transacciones. Ethereum extiende esta idea con *Merkle-Patricia tries*, que además de las transacciones permiten probar el estado de una cuenta o el resultado de una ejecución (Sección 2.2); esta capacidad de generar pruebas de inclusión compactas y verificables es, precisamente, el mecanismo que este trabajo utiliza para certificar *on-chain*, mediante una prueba de conocimiento cero, que una transacción de bloqueo fue efectivamente incluida en la cadena de origen.

2.1.3. Mecanismos de consenso

Que cada nodo mantenga una copia del registro no es suficiente: en una red sin autoridad central, distintos nodos pueden recibir transacciones en distinto orden o proponer bloques en simultáneo, y es necesario que todos converjan a una única secuencia de bloques válida. Este es un caso particular del problema de los generales bizantinos [6]: lograr acuerdo entre participantes distribuidos cuando una fracción de ellos puede ser maliciosa o fallar arbitrariamente. Un **mecanismo de consenso** es el protocolo que resuelve este problema definiendo, ante bloques o cadenas en conflicto, cuál es la versión canónica.

Los dos mecanismos dominantes son:

- **Proof-of-Work (PoW)**: introducido por Bitcoin [7], exige que quien propone un bloque resuelva un problema computacional costoso (encontrar un *nonce* tal que $H(\text{header})$ sea menor a un umbral de dificultad). La cadena válida es la que acumula mayor trabajo computacional total, lo que hace que reescribir el historial requiera superar la capacidad de cómputo de toda la red honesta. Su finalidad es *probabilística*: la confianza en que un bloque no será revertido crece con la cantidad de bloques confirmados sobre él, pero nunca es absoluta.

- **Proof-of-Stake (PoS)**: sustituye el costo computacional por un costo económico. Los participantes (*validadores*) bloquean una cantidad de la criptomoneda nativa como garantía y son seleccionados para proponer y atestiguar bloques en proporción a su participación bloqueada (*stake*). Un comportamiento malicioso detectable (como firmar dos bloques en conflicto para la misma posición) resulta en la pérdida de parte o la totalidad de esa garantía (*slashing*). A diferencia de PoW, las variantes modernas de PoS —como la que adoptó Ethereum tras *The Merge*— pueden alcanzar **finalidad determinística**: una vez que un bloque acumula suficientes atestiguaciones, revertirlo es no solo improbable sino que exige destruir económicamente a los validadores responsables. Este mecanismo de finalidad, y el comité que la certifica en Ethereum, se retoma en la Sección 2.2 y es la raíz de confianza sobre la que se construye el puente propuesto en este trabajo.

2.1.4. Roles de nodos y topología de red

No todos los participantes de una red blockchain cumplen la misma función ni requieren los mismos recursos:

- **Nodos completos** (*full nodes*): descargan y validan de forma independiente cada bloque y cada transacción desde el bloque génesis, manteniendo una copia íntegra del estado de la red. No confían en ningún tercero: verifican por sí mismos que se respetaron las reglas de consenso y de validez de transacciones.

- **Mineros o validadores**: además de validar, participan activamente en la producción de nuevos bloques (resolviendo el rompecabezas de PoW o siendo seleccionados y atestiguando en PoS).

- **Clientes ligeros** (*light clients*): no almacenan la cadena completa ni reejecutan todas las transacciones; en su lugar, confían en los encabezados de bloque —en particular en su raíz de Merkle y, en redes PoS, en la firma del comité que certifica su finalidad— para verificar de forma eficiente si una transacción específica fue incluida, mediante una prueba de inclusión como la descrita en la Sección 2.1.2. Este es precisamente el modelo de confianza que adopta el puente diseñado en este trabajo, cuyos contratos en la cadena de

destino no reejecutan la cadena de origen, sino que verifican pruebas de conocimiento cero de la firma del comité de sincronización de Ethereum sobre un bloque y de la inclusión de la transacción en ese bloque. El *relayer off-chain* se limita a generar dichas pruebas y a remitirlas a esos contratos.

Todos estos roles se comunican mediante una red *peer-to-peer* de propagación por *gossip*, en la que cada nodo retransmite a sus pares los bloques y transacciones que recibe y considera válidos, sin que exista un servidor central que coordine la difusión de la información.

2.2. Ethereum (Estructura de bloques)

2.3. Aztec (Estructura de bloques)

2.4. Puentes

CAPÍTULO 3

CONOCIMIENTO CERO

3.1. Fundamentos

El capítulo anterior describió los mecanismos mediante los cuales una red blockchain garantiza la integridad de su registro, desde el encadenamiento de bloques por hash hasta las pruebas de inclusión sobre árboles de Merkle y el modelo de confianza de los clientes ligeros (Sección 2.1). En todos esos casos, quien verifica una afirmación sobre la cadena necesita disponer de los datos pertinentes y ejecutar por sí mismo el cómputo correspondiente. Los sistemas de prueba estudiados en este capítulo, conocidos genéricamente como pruebas de conocimiento cero, ofrecen una alternativa a este modelo, puesto que permiten convencer a un verificador de que una afirmación es verdadera, o de que una computación se ejecutó correctamente, sin que este deba repetir dicha computación. Esta última capacidad no se deriva de la propiedad de conocimiento cero, sino de la sucintez de los sistemas empleados, distinción que se precisa en las Secciones 3.1.1 y 3.1.5.

La presente sección no pretende ofrecer un tratamiento exhaustivo de la teoría de las pruebas de conocimiento cero, sino establecer el vocabulario mínimo necesario para comprender las herramientas analizadas en el resto del capítulo. Para un desarrollo completo de los aspectos teóricos se remite al lector a la obra de Thaler [8], cuya terminología se adopta en lo que sigue.

3.1.1. Concepto de conocimiento cero

En su acepción habitual, demostrar una afirmación consiste en presentar evidencia que permita a otra parte comprobar su veracidad. Si alguien afirma conocer un valor w cuyo hash criptográfico coincide con un valor público y, es decir, un valor tal que $H(w) = y$, la forma más directa de demostrarlo es entregar w para que el receptor calcule $H(w)$ y lo compare con y. Esta demostración resulta convincente, pero el receptor termina conociendo el valor que

se pretendía mantener en secreto. Goldwasser, Micali y Rackoff demostraron que es posible convencer a otra parte de una afirmación de este tipo sin transferirle ningún conocimiento adicional a la validez de la afirmación misma, y denominaron a este tipo de demostraciones *pruebas de conocimiento cero* [9]. Posteriormente, Goldreich, Micali y Wigderson demostraron que, bajo supuestos criptográficos estándar, toda afirmación cuya veracidad puede comprobarse eficientemente a partir de un certificado admite una prueba de este tipo [10].

El ejemplo anterior permite introducir los cinco términos que se emplean en el resto del capítulo.

- El **probador** (*prover*), denotado $\mathcal{P}$, es la parte que afirma la validez de un enunciado y genera la evidencia que la respalda.
- El **verificador** (*verifier*), denotado $\mathcal{V}$, es la parte que examina dicha evidencia y decide si la acepta o la rechaza. Se asume que el verificador dispone de recursos de cómputo limitados, por lo que su trabajo debe ser eficiente.
- El **enunciado** (*statement*), denotado x, es la afirmación que se desea demostrar, expresada a partir de datos públicos conocidos por ambas partes. En el ejemplo, el enunciado es que existe un valor cuyo hash es y.
- El **testigo** (*witness*), denotado w, es la información que el probador utiliza para demostrar el enunciado y que no necesita entregar al verificador. En el ejemplo, el testigo es la preimagen w.
- La **prueba** (*proof*), denotada π, es el objeto que el probador genera a partir del enunciado y del testigo, y que el verificador examina para tomar su decisión.

Una **relación** $\mathcal{R}$ es un conjunto de pares (x, w) cuya pertenencia puede comprobarse eficientemente, y un enunciado x es verdadero cuando existe algún testigo w tal que $(x, w) \in \mathcal{R}$ [11]. En el ejemplo, la relación está formada por los pares (y, w) que satisfacen $H(w) = y$. Las herramientas que se analizan en las secciones siguientes reflejan esta distinción mediante las nociones de *entradas públicas*, que forman parte del enunciado, y *entradas privadas*, que forman parte del testigo.

Conviene aclarar que el conocimiento cero y la privacidad no son necesariamente lo mismo en las aplicaciones blockchain. Algunas aplicaciones emplean estas pruebas para ocultar información, como Zerocash, que valida pagos sin revelar su origen, su destino ni su monto [12]. Sin embargo, los sistemas de prueba modernos ofrecen además una propiedad independiente del conocimiento cero, que consiste en la posibilidad de verificar una computación en un tiempo muy inferior al que requiere ejecutarla. Walfish y Blumberg describen este objetivo como la verificación de computaciones sin reejecutarlas, de manera que una parte con recursos limitados puede delegar un cómputo en otra parte no confiable y comprobar después, a bajo costo, que el resultado recibido es correcto [13]. En muchas aplicaciones blockchain, los datos involucrados

son públicos y el valor de la prueba reside precisamente en esta segunda propiedad. La literatura y la industria emplean de todos modos la expresión *prueba de conocimiento cero* para referirse a estos sistemas, convención que se adopta también en el presente trabajo.

Un ejemplo relacionado con el Capítulo 2 permite apreciar esta diferencia. Una prueba de inclusión en un árbol de Merkle (Sección 2.1.2) puede formularse como un enunciado, cuyo contenido es que un elemento t pertenece al árbol de raíz r, y un testigo, formado por los hashes hermanos del camino que une a t con r. Ninguno de estos datos es secreto, ya que todos pueden obtenerse de la cadena. El interés de expresar esta verificación como un enunciado reside en la posibilidad de combinarla con otras condiciones de validez, como la comprobación de que la raíz r pertenece a un encabezado certificado por los validadores de la cadena, y de demostrar el conjunto mediante una única prueba cuya verificación resulte económica.

3.1.2. Propiedades fundamentales

Un sistema de prueba de conocimiento cero se caracteriza por tres propiedades, cada una de las cuales protege a una de las partes frente a un comportamiento incorrecto de la otra [9, 8].

La **completitud** (*completeness*) garantiza que, si el enunciado es verdadero y el probador sigue el protocolo utilizando un testigo válido, el verificador acepta la prueba. Esta propiedad protege al probador honesto, puesto que asegura que una demostración correcta no será rechazada.

La **solidez** (*soundness*) garantiza que, si el enunciado es falso, ningún probador logra que el verificador acepte, salvo con una probabilidad *despreciable*, es decir, tan pequeña que en la práctica puede considerarse nula. Esta propiedad protege al verificador frente a un probador deshonesto, denotado $\mathcal{P}^*$, que intenta hacer pasar por verdadera una afirmación falsa. Cuando esta garantía depende de la dificultad de algún problema computacional, como ocurre en todos los sistemas analizados en este capítulo, el sistema se denomina técnicamente *argumento* en lugar de *prueba* [14].

En ciertos casos la solidez resulta insuficiente. En el ejemplo de la preimagen, prácticamente todo valor y posee alguna preimagen, de manera que demostrar su existencia carece de interés. Lo que se pretende demostrar es que el probador *conoce* una. Esta exigencia se denomina **solidez de conocimiento** (*knowledge soundness*) y establece que de todo probador capaz de convencer al verificador podría obtenerse un testigo válido, lo que implica que dicho probador efectivamente lo conocía [15]. Los sistemas que cumplen esta propiedad se denominan *argumentos de conocimiento*.

El **conocimiento cero** (*zero-knowledge*) garantiza que el verificador no obtiene de la prueba ninguna información sobre el testigo más allá de la validez del enunciado. Esta propiedad protege al probador, y su formalización se basa en mostrar que todo lo que el verificador observa podría haberse generado a partir del enunciado sin conocer el testigo [9].

La importancia relativa de cada propiedad depende de la aplicación. En un puente entre

cadenas, el verificador es un contrato inteligente que acuña activos cuando acepta una prueba, y el adversario principal es un probador que intenta obtener esa acuñación a partir de un evento que nunca ocurrió en la cadena de origen. En este escenario la propiedad crítica es la solidez, puesto que su violación permitiría acuñar activos sin respaldo. El conocimiento cero, en cambio, resulta secundario cuando los datos que se prueban son públicos.

3.1.3. Pruebas interactivas y no interactivas

En el modelo original de Goldwasser, Micali y Rackoff, probador y verificador intercambian varios mensajes antes de que el verificador tome su decisión [9]. El verificador envía *desafíos* elegidos al azar, y la solidez se apoya en que el probador no puede anticiparlos. Este tipo de protocolos se denomina **interactivo** y presenta limitaciones importantes para su uso en una red blockchain. La interacción requiere que ambas partes participen sucesivamente en cada ronda, aunque no necesariamente mediante una conexión simultánea, y debe repetirse con cada verificador. Además, la transcripción resultante no constituye, en general, una prueba verificable por terceros, ya que estos no pueden comprobar que los desafíos se eligieron al azar y no en acuerdo con el probador. En un protocolo de conocimiento cero, de hecho, todo lo que observa el verificador podría haberse generado sin conocer el testigo (Sección 3.1.2), aunque la transferibilidad de una transcripción depende, en última instancia, del protocolo concreto. En el caso de un contrato inteligente, que se ejecuta de manera determinista como respuesta a cada transacción, un protocolo de varias rondas podría implementarse mediante una secuencia de transacciones, pero a costa de gas adicional, de latencia entre rondas, de la necesidad de conservar estado entre transacciones y de la dificultad de obtener desafíos aleatorios que el probador no pueda manipular, lo que incrementa además la complejidad de la implementación.

Una prueba **no interactiva** consiste, en cambio, en un único mensaje π que el probador genera por sí solo y que cualquier parte puede verificar en cualquier momento. Blum, Feldman y Micali demostraron que este tipo de pruebas es posible cuando ambas partes comparten ciertos parámetros públicos generados de antemano mediante un procedimiento denominado **configuración inicial** (*setup*) [16]. Si ese procedimiento emplea valores secretos que deben destruirse, porque su conocimiento permitiría generar pruebas falsas, se habla de una **configuración confiable** (*trusted setup*). Si solo emplea aleatoriedad pública, la configuración se denomina **transparente**. Esta distinción es una de las diferencias principales entre las familias de sistemas presentadas en la Sección 3.1.5.

La técnica más difundida para eliminar la interacción es la **transformación de Fiat-Shamir** [17], aplicable a protocolos de *desafíos públicos*, es decir, aquellos en los que el verificador se limita a enviar valores aleatorios. La transformación reemplaza cada desafío por el hash de la transcripción previa, que debe incluir el enunciado público y los mensajes enviados hasta ese momento por el probador. Un probador deshonesto puede, sin embargo, calcular el hash de distintos mensajes candidatos antes de elegir cuál enviar, en busca de uno que le proporcione un desafío favorable, estrategia conocida como ataque de búsqueda (*grinding*). La seguridad de

la transformación suele analizarse en el modelo de oráculo aleatorio, que idealiza la función de hash como una función perfectamente aleatoria [18], y en ese modelo el éxito de dicha búsqueda no depende únicamente del tamaño del espacio de desafíos, sino también de la solidez del protocolo original, de sus parámetros y del número de evaluaciones de hash que el adversario puede realizar. Thaler ilustra este punto señalando que, si el protocolo original ofrece solo 60 bits de seguridad, unas 2^{60} evaluaciones de hash bastan para encontrar un mensaje favorable, cantidad alcanzable con hardware actual, y recomienda configurar el protocolo con un margen holgadamente superior a 80 bits [8, Sección 5.3]. Por estos motivos, la transformación no resulta automáticamente segura para cualquier protocolo ni para cualquier forma de construir la transcripción. Omitir el enunciado del cálculo del desafío, por ejemplo, ha dado lugar a ataques documentados [19]. Aplicada correctamente, la transformación permite generar la prueba sin intervención del verificador. Una prueba así obtenida puede incluirse en una transacción y ser verificada tanto por un contrato inteligente como por cualquier nodo que valide el bloque, lo que la convierte en un objeto persistente y verificable públicamente.

3.1.4. De una computación a una prueba

Esta subsección describe en primer lugar el enfoque basado en circuitos aritméticos y restricciones, que constituye el fundamento de herramientas como Circom y Noir. En este enfoque, el desarrollador describe la computación mediante un lenguaje, como Circom o Noir, y un compilador la transforma en la representación matemática sobre la que opera el sistema de prueba. La construcción de esa representación se denomina **aritmetización** [8]. Todos los valores que intervienen en esa representación son elementos de un **cuerpo finito** $\mathbb{F}_p$, formado por los enteros $\{0, 1, \ldots, p-1\}$ con la suma y el producto calculados módulo un número primo p, de manera que, a diferencia de lo que ocurre en un procesador convencional, todo resultado se reduce módulo p. Esta descripción no se aplica de igual modo a todos los sistemas de prueba. Las zkVM, presentadas al final de esta subsección, no requieren que el desarrollador exprese la computación de esta forma y emplean internamente representaciones distintas, que se analizan en la Sección 3.4.

Circuitos y restricciones. Un **circuito aritmético** representa la computación como una red de compuertas de suma y de producto sobre $\mathbb{F}_p$. Sin embargo, un sistema de prueba no necesita repetir la computación, sino comprobar que un conjunto de valores propuesto por el probador es coherente con ella. Por ese motivo, el circuito se expresa como un **sistema de restricciones**, es decir, un conjunto de ecuaciones sobre $\mathbb{F}_p$ que deben cumplirse simultáneamente y cuyas incógnitas incluyen las entradas, las salidas y todos los valores intermedios. Para demostrar que se conoce un valor w tal que $w^3 + w + 5 = 35$, por ejemplo, la computación puede descomponerse en tres restricciones con dos variables intermedias v_1 y v_2.

$$w \cdot w = v_1, \qquad v_1 \cdot w = v_2, \qquad v_2 + w + 5 = 35. \tag{3.1}$$

El testigo que satisface este sistema no es solamente el valor $w = 3$, sino la asignación completa $(w, v_1, v_2) = (3, 9, 27)$. En general, el testigo de un circuito está formado por todos los valores que surgen al ejecutar la computación. El formato de restricciones más difundido es el sistema de restricciones de rango uno (*Rank-1 Constraint System*, R1CS) [20], que se retoma en la Sección 3.2.

Esta forma de representación tiene dos consecuencias prácticas. La primera es que una restricción omitida no produce un error de ejecución, sino que amplía el conjunto de testigos aceptados, lo que permite a un probador deshonesto satisfacer el sistema con valores que no corresponden a la computación prevista. Los circuitos insuficientemente restringidos constituyen, de hecho, una categoría recurrente entre las 141 vulnerabilidades reales analizadas por Chaliasos et al. [21]. La segunda consecuencia es que las operaciones que no son nativas del cuerpo finito resultan costosas. Las funciones de hash SHA-256 y Keccak-256, empleadas en el consenso y en las estructuras de datos de Ethereum, operan sobre bits, y cada bit debe representarse como un elemento de $\mathbb{F}_p$ restringido a los valores 0 y 1, por lo que una única evaluación de estas funciones requiere un número de restricciones muy superior al de una operación aritmética nativa. Los sistemas actuales mitigan este costo mediante tablas de búsqueda (*lookup tables*), técnica que se retoma en la Sección 3.3.

Generación y verificación de la prueba. Una vez definida la representación, el proceso completo se desarrolla en cuatro etapas, que la Figura 3.1 representa de manera esquemática.

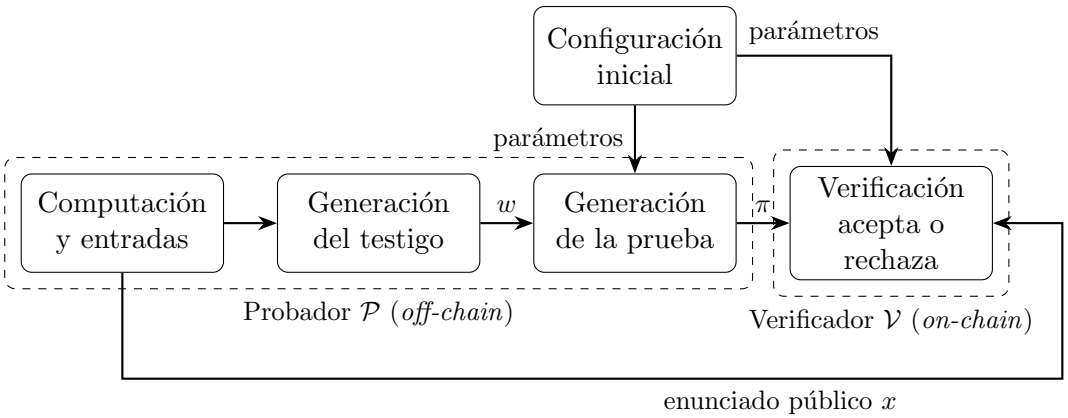


Figura 3.1: Etapas de generación y verificación de una prueba.

1. **Configuración inicial.** A partir del sistema de restricciones se generan los parámetros públicos que emplean el probador y el verificador. Esta etapa se realiza una única vez y, en los sistemas transparentes, no requiere valores secretos.

2. **Generación del testigo.** El probador ejecuta la computación sobre las entradas concretas y registra todos los valores que intervienen en ella.

3. **Generación de la prueba.** El probador transforma el testigo en una prueba π mediante operaciones criptográficas. En muchas construcciones prácticas, esta etapa suele ser una

de las más costosas del proceso, tanto en tiempo como en memoria.

4. **Verificación.** El verificador comprueba la prueba π a partir del enunciado x y de los parámetros públicos, sin acceso al testigo ni a la computación original, y la acepta o la rechaza.

El reparto del trabajo entre ambas partes es marcadamente asimétrico. El costo de generar la prueba crece al menos de manera lineal con el tamaño de la computación y, en la práctica, suele superar ampliamente el de ejecutarla, mientras que el costo de verificarla es, en los sistemas que se describen en la Sección 3.1.5, constante o muy pequeño en comparación. Por ese motivo, en las aplicaciones blockchain la prueba se genera fuera de la cadena (*off-chain*), en equipos con recursos suficientes, y se verifica dentro de la cadena (*on-chain*), en un contrato inteligente que recibe la prueba y el enunciado como datos de una transacción. Conviene subrayar, además, que la garantía obtenida se limita a que el probador conoce un testigo que satisface las restricciones para el enunciado dado. Su valor depende, por lo tanto, de que las restricciones expresen fielmente la afirmación que se pretende establecer, y alcanza únicamente a los valores que forman parte del enunciado, de modo que todo dato que el verificador utilice para tomar su decisión, como el destinatario o el monto de una transferencia, debe ser una entrada pública o estar vinculado criptográficamente a una.

Del circuito a la ejecución de programas. En el enfoque descrito, cada computación da lugar a un sistema de restricciones específico, cuyo tamaño queda fijado de antemano. Los ciclos deben desplegarse con una cantidad máxima de iteraciones conocida al construir el circuito, las bifurcaciones que dependen de los datos obligan a representar ambas ramas, y toda modificación de la lógica produce un circuito distinto. Circom y Noir, analizados en las Secciones 3.2 y 3.3, pertenecen a este enfoque, aunque difieren en el nivel de abstracción que ofrecen al desarrollador.

Existe una forma conceptualmente distinta de plantear el problema. En lugar de demostrar que un testigo satisface las restricciones de una computación particular, puede demostrarse que un programa arbitrario, ejecutado sobre un procesador determinado, produjo cierto resultado. En este caso las restricciones no describen el programa sino el procesador, puesto que expresan que cada paso de la ejecución es una transición válida de acuerdo con la instrucción correspondiente. El testigo pasa a ser la **traza de ejecución**, es decir, el registro del estado del procesador en cada paso, y el programa se convierte en un dato más del enunciado, de manera que un único sistema de restricciones permite probar la ejecución de cualquier programa. Ben-Sasson et al. materializaron esta idea al construir pruebas sucintas para la ejecución de programas escritos en el lenguaje C [22].

Una **máquina virtual de conocimiento cero** (*zero-knowledge virtual machine*, zkVM) aplica este principio a un conjunto de instrucciones concreto. La zkVM ejecuta un programa escrito en un lenguaje de propósito general y compilado con herramientas convencionales, registra su traza y genera una prueba de que la ejecución fue correcta y produjo determinadas salidas públicas. SP1, analizado en la Sección 3.4, ejecuta de este modo programas escritos en Rust

y compilados al conjunto de instrucciones RISC-V [4]. La diferencia entre ambos enfoques no reside en lo que pueden demostrar, sino en los compromisos que adoptan. En el enfoque basado en circuitos, el desarrollador razona en términos de restricciones, lo que exige un conocimiento especializado pero permite ajustar la representación a la computación. En el enfoque basado en una zkVM, el desarrollador razona en términos de un programa convencional, lo que facilita el desarrollo, la reutilización de bibliotecas y la depuración, a cambio de un costo adicional por cada instrucción ejecutada. Este compromiso es el que motiva la comparación entre Noir y SP1 planteada en la hipótesis del presente trabajo.

3.1.5. SNARKs y STARKs

Las propiedades descritas en la Sección 3.1.2 no se refieren al tamaño de la prueba ni al costo de verificarla, aspectos determinantes en las aplicaciones blockchain, ya que la prueba debe transmitirse en una transacción y su verificación se paga en gas. Un sistema de prueba es **sucinto** cuando tanto el tamaño de la prueba como el tiempo de verificación son muy pequeños en comparación con el tamaño de la computación probada [8]. Las dos familias de sistemas sucintos más utilizadas en la práctica son los SNARKs y los STARKs.

Ambas familias comparten una idea común, que consiste en codificar la computación como relaciones entre polinomios y comprobarlas en unos pocos puntos elegidos al azar, lo que se fundamenta en el principio de Schwartz–Zippel [23, 24] y reduce drásticamente el trabajo del verificador. Se diferencian principalmente en el mecanismo con el que el probador se compromete con esos polinomios, basado en curvas elípticas en los SNARKs más difundidos, como ocurre con el esquema KZG [25], y en funciones de hash en los STARKs, mediante árboles de Merkle y el protocolo FRI [26].

SNARKs. La sigla SNARK (*Succinct Non-interactive ARgument of Knowledge*) describe un argumento sucinto, no interactivo y con solidez de conocimiento [27]. Un SNARK que además es de conocimiento cero se denomina zk-SNARK. Las construcciones más difundidas se basan en curvas elípticas y emplean una operación criptográfica denominada *emparejamiento bilineal*, definida entre grupos asociados a dichas curvas. El sistema de Groth, conocido como Groth16, produce pruebas del orden de pocos cientos de bytes, cuya verificación requiere un número constante de emparejamientos más un trabajo proporcional al número de entradas públicas, ya que cada una de ellas debe combinarse con la clave de verificación mediante una operación sobre la curva [28, 29]. Para un número fijo de entradas públicas y un parámetro de seguridad dado, el costo de verificación no depende, por lo tanto, del tamaño del circuito. PLONK, otro sistema de esta familia, admite una configuración universal, válida para cualquier circuito hasta un tamaño máximo, en lugar de la configuración específica para cada circuito que requiere Groth16 [30]. Noir, el lenguaje analizado en la Sección 3.3, delega la generación de pruebas en un backend independiente. Su backend de referencia, Barretenberg, genera pruebas mediante UltraHonk, un sistema de la familia de PLONK [3, 31].

La principal limitación de construcciones como Groth16, o de PLONK instanciado con compromisos KZG, es que requieren una configuración confiable, cuyos valores secretos permitirían generar pruebas falsas si no se destruyen. Este requisito depende de la construcción y no es propio de todos los SNARKs, ya que existen esquemas basados en compromisos transparentes que no necesitan una ceremonia de este tipo [8]. Para reducir esta confianza se realizan ceremonias con numerosos participantes, cuyo resultado es seguro siempre que al menos uno de ellos haya destruido su contribución [32]. En contrapartida, estos sistemas ofrecen la verificación más económica disponible en Ethereum, que desde 2017 incorpora contratos precompilados para las operaciones sobre la curva elíptica `alt_bn128`, también denominada BN254 [33, 34].

STARKs. La sigla STARK (*Scalable Transparent ARgument of Knowledge*) designa una familia de sistemas propuesta por Ben-Sasson et al. que prescinde de la configuración confiable [35]. En lugar de curvas elípticas, los STARKs se basan únicamente en funciones de hash resistentes a colisiones, lo que los hace además plausiblemente resistentes a ataques mediante computación cuántica. Son *transparentes*, en el sentido descrito en la Sección 3.1.3, y *escalables*, ya que el tiempo de generación de la prueba crece de manera casi lineal con el tamaño de la computación, lo que resulta conveniente para computaciones extensas como la ejecución de un programa completo. Su principal desventaja es el tamaño de la prueba, varios órdenes de magnitud mayor que el de una prueba Groth16, lo que encarece considerablemente su verificación directa en Ethereum. La Tabla 3.1 resume las diferencias entre ambas familias.

Tabla 3.1: Diferencias conceptuales entre SNARKs y STARKs.

Característica	**SNARK**	**STARK**
Tamaño de la prueba	Muy pequeño	Mayor
Verificación	Muy eficiente	Eficiente, pero generalmente más costosa
Configuración inicial	Algunas construcciones requieren una configuración confiable	Transparente
Base criptográfica	Habitualmente curvas elípticas y emparejamientos	Funciones de hash y FRI
Contexto de uso	Circuitos y verificación compacta	Computaciones extensas y sistemas de tipo zkVM

Ambas familias no son excluyentes. Es posible verificar una prueba STARK dentro de un circuito y generar sobre esa verificación una prueba SNARK, de manera que el resultado combine la escalabilidad del probador de los STARKs con la verificación económica de los SNARKs. Esta composición es la que emplean las zkVM modernas para que sus pruebas puedan verificarse en Ethereum, y su aplicación en SP1 se analiza en la Sección 3.4.

Relación con los puentes entre cadenas. Los conceptos presentados permiten reformular el problema de los puentes descrito en la Sección 2.4. Un contrato inteligente desplegado en la cadena de destino necesita establecer la validez de una afirmación sobre otra cadena, por ejemplo, que una transacción de bloqueo fue incluida en un bloque certificado por los validadores de la cadena de origen. Un cliente ligero (Sección 2.1.4) establecería esta afirmación verificando la firma que certifica el encabezado y la prueba de inclusión de la transacción, pero reproducir esas verificaciones dentro de un contrato inteligente resulta demasiado costoso. Si ambas verificaciones se expresan como un único enunciado, el trabajo costoso queda a cargo de un probador fuera de la cadena, y el contrato solo necesita verificar una prueba sucinta. La solidez del sistema garantiza entonces que ningún probador puede obtener una prueba válida de un enunciado falso respecto de la relación compilada, de modo que la aceptación del contrato deja de depender de la honestidad de un intermediario. Esa garantía no agota, sin embargo, la seguridad del puente, que depende también de que las restricciones expresen fielmente la afirmación prevista, de los supuestos de seguridad y de la finalidad del consenso de la cadena de origen, de la vinculación criptográfica entre la prueba y los datos públicos que utiliza el contrato receptor, de la corrección de la lógica de ese contrato y de su protección contra la repetición de una misma operación. Xie et al. demostraron la viabilidad práctica de este enfoque [36]. El diseño concreto de esta verificación para el puente desarrollado en el presente trabajo se expone en el Capítulo 4.

3.2. Circom

La Sección 3.1.4 presentó el enfoque basado en circuitos aritméticos y restricciones, en el cual la computación que se desea probar se expresa como un conjunto de ecuaciones sobre un cuerpo finito. Circom constituye una de las herramientas más difundidas dentro de ese enfoque y permite observar en concreto cómo una descripción de la computación se transforma en un sistema de restricciones, cómo se obtiene el testigo correspondiente y cómo ambos elementos se utilizan para generar una prueba. La presente sección describe el lenguaje y su compilador, el proceso que conduce desde un circuito hasta una prueba verificable en Ethereum y las limitaciones que este modelo de desarrollo impone, en particular cuando se aplica a verificaciones propias de las redes blockchain.

3.2.1. Origen y propósito

Circom es un lenguaje para la descripción de circuitos aritméticos, acompañado de un compilador que traduce dichas descripciones a un sistema de restricciones de rango uno (R1CS). Fue desarrollado en el ecosistema del proyecto iden3, y su diseño y semántica fueron descritos académicamente por Bellés-Muñoz et al. [37]. La versión actual del compilador, conocida como Circom 2, está implementada en Rust y reemplazó a una primera implementación en JavaScript que se mantiene congelada [38]. El ecosistema se completa con circomlib, una biblioteca de

plantillas reutilizables que implementan primitivas criptográficas como las funciones de hash Poseidon, MiMC y Pedersen, firmas EdDSA, comparadores y árboles de Merkle dispersos [39], y con snarkjs, una implementación en JavaScript y WebAssembly de sistemas de prueba que consume los artefactos producidos por el compilador [29].

Circom es un **lenguaje de dominio específico** (*domain-specific language*, DSL), es decir, un lenguaje cuyas construcciones corresponden directamente a los elementos de un dominio de problemas concreto, en lugar de ofrecer las abstracciones generales de un lenguaje de programación convencional. En este caso, los elementos del dominio son las señales, que representan valores de $\mathbb{F}_p$, y las restricciones que dichas señales deben satisfacer. Esta orientación determina la principal diferencia entre escribir un programa convencional y describir un circuito en Circom. En un programa convencional, las instrucciones se ejecutan secuencialmente y su efecto queda determinado por el resultado de esa ejecución. En un circuito de Circom, en cambio, la descripción cumple dos funciones distintas. Por un lado, indica cómo calcular los valores de las señales a partir de las entradas, cálculo que solo realiza el probador. Por otro, establece las ecuaciones que esos valores deben satisfacer, y únicamente estas ecuaciones forman parte de lo que el verificador comprueba. La separación entre ambas funciones, que Circom expone de forma explícita al desarrollador, es el aspecto central del lenguaje y la fuente de sus principales riesgos, analizados en la Sección 3.2.4.

Circom no constituye por sí mismo un sistema de prueba. Su compilador produce el sistema de restricciones y un programa capaz de calcular el testigo, pero la configuración inicial, la generación de la prueba y su verificación corresponden a herramientas externas, como snarkjs, que implementan los sistemas descritos en la Sección 3.1.5.

3.2.2. Construcción de circuitos

La unidad básica de descripción en Circom es la **plantilla** (*template*), que define un subcircuito parametrizable con un conjunto de señales y las restricciones que las vinculan. Una plantilla puede instanciarse como **componente** dentro de otra plantilla, lo que permite construir circuitos complejos a partir de piezas más simples, del mismo modo en que circomlib ofrece plantillas para funciones de hash o comparaciones que se integran en circuitos mayores. Todo circuito tiene un componente principal, denominado `main`, que define la interfaz del circuito completo [38].

Las **señales** son los valores sobre los que operan las restricciones y se clasifican en señales de entrada (`signal input`), de salida (`signal output`) e intermedias (`signal`). Su visibilidad se relaciona directamente con las nociones de enunciado y testigo de la Sección 3.1.1. Las señales de salida del componente principal son siempre públicas, las señales de entrada son privadas salvo que se declaren explícitamente como públicas mediante la sintaxis `component main {public [...]}`, y las señales intermedias son siempre privadas [38]. Las señales públicas forman, por lo tanto, el enunciado que conoce el verificador, mientras que los valores de todas las señales conforman, conceptualmente, el testigo. Circom admite además variables declaradas con `var`

que, a diferencia de las señales, no forman parte del testigo ni generan restricciones por sí mismas. Una variable puede contener valores conocidos durante la compilación, como índices o contadores, pero también valores que dependen de señales y que, por lo tanto, solo se conocen durante el cálculo del testigo, así como expresiones aritméticas que el compilador emplea para construir restricciones [38].

La relación entre señales se establece mediante tres operadores, cuya diferencia es esencial para comprender el lenguaje [38]. El operador `===` impone una restricción entre dos expresiones sin asignar ningún valor. El operador `<--` asigna un valor a una señal sin generar ninguna restricción, de modo que ese valor solo interviene en el cálculo del testigo. El operador `<==` combina ambas acciones, ya que asigna el valor y genera simultáneamente la restricción de igualdad correspondiente. Los operadores `==>` y `-->` son las variantes de `<==` y `<--` con la asignación escrita en sentido inverso. El compilador exige que toda restricción sea de grado a lo sumo dos y pueda llevarse a la forma $A \cdot B + C = 0$, donde A, B y C son combinaciones lineales de señales, y rechaza las restricciones que no admiten esa forma [38]. Esta forma coincide con la de una fila de un sistema R1CS, en el que cada restricción relaciona el producto de dos combinaciones lineales con una tercera (Sección 3.1.4). Las restricciones lineales se representan en este formato tomando una constante como uno de los factores, mientras que no toda expresión de grado dos admite una única fila. La restricción $e = a \cdot b + c \cdot d$, por ejemplo, es rechazada por el compilador y debe descomponerse mediante una señal intermedia en dos restricciones. Cada restricción de Circom tiene, por lo tanto, la forma de una fila de dicho sistema, aunque el compilador puede simplificar y transformar el conjunto de restricciones antes de producirlo.

La Figura 3.2 muestra un ejemplo mínimo que retoma la computación utilizada en la Sección 3.1.4. El circuito permite demostrar el conocimiento de un valor privado w tal que $w^3 + w + 5 = y$, siendo y una salida pública.

```
1  pragma circom 2.1.6;
2
3  template Cubic() {
4      signal input w;      // entrada privada
5      signal output y;     // salida publica
6      signal v1;           // senales intermedias
7      signal v2;
8
9      v1 <== w * w;
10     v2 <== v1 * w;
11     y  <== v2 + w + 5;
12 }
13
14 component main = Cubic();
```

Figura 3.2: Circuito Circom que expresa la relación $w^3 + w + 5 = y$.

Cada uno de los operadores `<==` del ejemplo expresa una restricción, de modo que el código

describe el sistema

$$w \cdot w - v_1 = 0, \qquad v_1 \cdot w - v_2 = 0, \qquad (v_2 + w + 5) \cdot 1 - y = 0, \tag{3.2}$$

cuyas dos primeras restricciones son no lineales y cuya tercera es lineal. Todas las operaciones se realizan en $\mathbb{F}_p$, donde p es, por defecto, el orden primo de 254 bits del grupo asociado a la curva BN254, la misma curva para la que Ethereum dispone de contratos precompilados [38, 34]. Al evaluar el circuito para la entrada $w = 3$ se obtienen los valores $(y, w, v_1, v_2) = (35, 3, 9, 27)$, que corresponden a la asignación presentada en la Sección 3.1.4. Solo y es una señal pública, por lo que su valor forma parte de las entradas públicas del sistema de prueba, mientras que w queda excluida de ellas. Esta exclusión determina únicamente qué valores recibe el verificador. Que la prueba no revele información adicional sobre w más allá de la que pueda deducirse del valor público y, depende de que el sistema de prueba empleado satisfaga la propiedad de conocimiento cero descrita en la Sección 3.1.2.

El sistema (3.2) reúne las restricciones expresadas en el código, que no coinciden necesariamente con las que conserva el sistema compilado. El compilador ofrece tres niveles de simplificación. La opción `--O0` no aplica ninguna, `--O1` solo simplifica igualdades entre señales y entre señales y constantes, y `--O2` aplica la simplificación completa, que puede eliminar restricciones lineales sustituyendo una señal por la expresión que la define [38]. El nivel aplicado por defecto depende de la versión del compilador, ya que fue `--O2` hasta la versión 2.1.9 y pasó a ser `--O1` a partir de la versión 2.2.0 [40]. La simplificación completa despeja, de cada restricción lineal que contenga una señal privada, una de esas señales y la sustituye en las demás restricciones, sin eliminar nunca las señales públicas [38]. En una compilación del ejemplo realizada para el presente trabajo con la versión 2.2.3, las opciones `--O0` y `--O1` conservaron las tres restricciones, mientras que `--O2` despejó $v_2 = y - w - 5$ de la restricción lineal y lo sustituyó en la segunda, de modo que el sistema resultante constó de las dos restricciones $w \cdot w = v_1$ y $v_1 \cdot w = y - w - 5$, en las que v_2 ya no interviene. La elección de la señal que se despeja corresponde al algoritmo del compilador, por lo que el sistema resultante es una de varias formas algebraicamente equivalentes de expresar la misma relación.

El ejemplo permite precisar qué significa que un circuito esté **correctamente restringido**. Un circuito lo está cuando las asignaciones que satisfacen su sistema de restricciones corresponden exactamente a los casos en que se cumple la relación que el desarrollador pretende demostrar. Esta condición no exige que todas las señales auxiliares queden determinadas unívocamente. En el circuito de la Figura 3.2, una vez fijado w, las restricciones determinan v_1, v_2 e y, de modo que ningún probador puede presentar un valor de y distinto de $w^3 + w + 5$. En la plantilla `IsZero` de la Sección 3.2.4, en cambio, cuando la entrada es nula las restricciones fijan la salida en 1 pero admiten cualquier valor de la señal auxiliar `inv`, libertad que no permite aceptar ninguna afirmación incorrecta. Un circuito *insuficientemente restringido* admite, por el contrario, asignaciones que conducen a aceptar afirmaciones que la especificación rechaza, mientras que un circuito *sobrerrestringido* rechaza asignaciones que la especificación considera

legítimas. Estos defectos deben distinguirse de las propiedades del sistema de prueba descritas en la Sección 3.1.2. El sistema criptográfico sigue siendo sólido y completo respecto de la relación que efectivamente se compiló, pero un circuito insuficientemente restringido compromete la solidez respecto de la especificación prevista, y uno sobrerrestringido impide demostrar afirmaciones que, según dicha especificación, deberían poder demostrarse.

3.2.3. Compilación, cálculo del testigo y generación de la prueba

El proceso que conduce desde un circuito de Circom hasta una prueba verificable concreta las etapas generales descritas en la Sección 3.1.4, y se distribuye entre el compilador de Circom y un sistema de prueba externo. La Figura 3.3 representa este proceso para el caso de Groth16 implementado en snarkjs.

En la primera etapa, el compilador procesa el circuito y genera dos artefactos independientes. El primero es el sistema de restricciones, almacenado en un archivo binario con extensión `.r1cs`, que define formalmente la relación que la prueba certificará. El segundo es un programa capaz de calcular el testigo, que el compilador genera en WebAssembly o en C++, siendo esta última opción más eficiente para circuitos de gran tamaño [38]. Durante la compilación, el compilador comprueba además que todas las restricciones sean de grado a lo sumo dos y admitan la forma descrita, y aplica las simplificaciones correspondientes al nivel de optimización elegido. Desde la versión 2.0.8 incorpora también una advertencia para los usos de `<--` en los que la expresión asignada es cuadrática y podría emplearse `<==` [40]. En la versión 2.2.3 utilizada en las compilaciones de este trabajo, dicha advertencia se emitió al activar la opción `--inspect`, que realiza comprobaciones adicionales destinadas a identificar señales potencialmente insuficientemente restringidas, como aquellas que no aparecen en ninguna restricción [38].

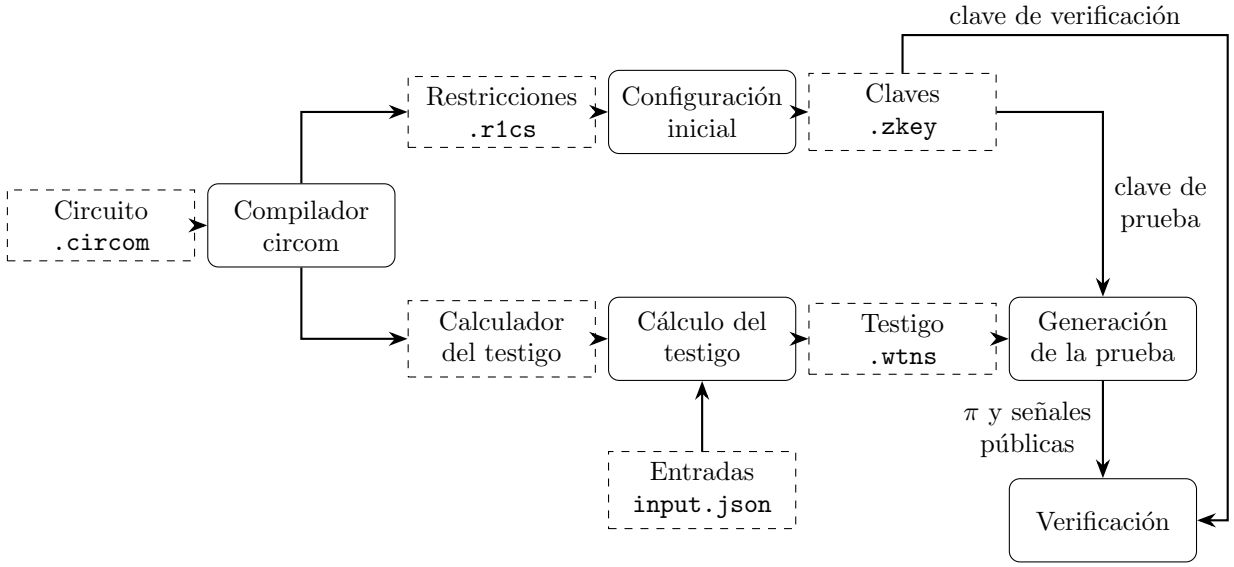


Figura 3.3: Proceso de compilación, cálculo del testigo, generación y verificación de una prueba a partir de un circuito Circom.

En la segunda etapa, el probador ejecuta el calculador del testigo sobre entradas concretas,

expresadas en un archivo en formato JSON, y obtiene un archivo `.wtns` con la asignación correspondiente al sistema R1CS compilado [38]. Este archivo contiene los valores de las señales que el sistema conserva, precedidos por la constante 1 que el formato emplea para representar términos independientes, por lo que su tamaño y su orden dependen de la compilación. En el ejemplo de la Figura 3.2, el testigo obtenido con `--O1` contiene los cinco valores $(1, 35, 3, 9, 27)$, mientras que con `--O2` la señal v_2 no forma parte del sistema y el testigo se reduce a cuatro valores. Este cálculo ejecuta las asignaciones descritas mediante los operadores `<--` y `<==`, pero no constituye una garantía criptográfica, ya que un probador puede producir un archivo de testigo por cualquier otro medio. Lo que determina si una prueba será aceptada no es el programa que calculó el testigo, sino el hecho de que dicho testigo satisfaga el sistema de restricciones.

La tercera etapa corresponde a la configuración inicial del sistema de prueba. En el caso de Groth16, cuyos fundamentos se presentaron en la Sección 3.1.5, la configuración se divide en dos fases. La primera, denominada *powers of tau*, es independiente del circuito y puede reutilizarse para cualquier circuito hasta un tamaño máximo. La segunda es específica del circuito y produce un archivo `.zkey` que contiene la clave de prueba, a partir del cual se exporta la clave de verificación [38]. Ambas fases admiten contribuciones de múltiples participantes, en el sentido de las ceremonias descritas en la Sección 3.1.5 [32]. snarkjs implementa también PLONK y FFLONK, que solo requieren la fase universal [29].

En la cuarta etapa, el probador combina la clave de prueba y el testigo para generar la prueba π, junto con un archivo que contiene los valores de las señales públicas. Finalmente, la verificación puede realizarse con snarkjs o mediante un contrato inteligente en Solidity que la propia herramienta genera a partir de la clave de verificación [29]. Dado que, por defecto, Circom trabaja sobre el cuerpo asociado a la curva BN254, el contrato generado puede apoyarse en los contratos precompilados de Ethereum para dicha curva [33, 34], lo que hace posible la verificación *on-chain* descrita en la Sección 3.1.4.

Este proceso permite delimitar las responsabilidades de cada componente. El compilador define la relación que se probará, el calculador del testigo obtiene una asignación que la satisface, el sistema de prueba genera a partir de ella una prueba sucinta y el verificador comprueba esa prueba. En el caso de Groth16, la decisión del verificador se basa en la clave de verificación, la prueba π y los valores de las entradas públicas. El verificador no recibe el archivo `.r1cs` ni ejecuta el programa que calculó el testigo, pero la clave de verificación se deriva del sistema de restricciones compilado durante la configuración inicial, por lo que la relación que la prueba certifica es exactamente la que dicho sistema expresa. El programa de generación del testigo no aporta, por sí mismo, ninguna garantía criptográfica, y cualquier propiedad que no haya quedado expresada como restricción carece de respaldo, con independencia de lo que haga ese programa.

3.2.4. Limitaciones y consideraciones de seguridad

La separación entre cálculo y restricción descrita en la Sección 3.2.1 es la principal fuente de errores en el desarrollo con Circom. El operador `<--` resulta necesario cuando el cálculo del valor de una señal no puede expresarse directamente mediante una restricción cuadrática, pero el valor así asignado solo queda respaldado por la prueba si el desarrollador añade restricciones que lo vinculen con las demás señales. La plantilla `IsZero` de circomlib, reproducida en la Figura 3.4, ilustra este patrón [39].

```
template IsZero() {
    signal input in;
    signal output out;
    signal inv;

    inv <-- in != 0 ? 1 / in : 0; // solo calculo
    out <== -in * inv + 1;         // asigna y restringe
    in * out === 0;                // solo restringe
}
```

Figura 3.4: Plantilla `IsZero` de circomlib, que combina un cálculo auxiliar con restricciones explícitas.

La plantilla devuelve 1 si la entrada es nula y 0 en caso contrario. Para una entrada distinta de cero, la relación $\texttt{in} \cdot \texttt{inv} = 1$ es cuadrática, pero la expresión que calcula `inv` es condicional y emplea una división, por lo que no puede asignarse mediante `<==`. Por ese motivo, la asignación `inv <-- in != 0 ? 1 / in : 0` solo calcula un valor candidato, que se incorpora al testigo, mientras que las dos restricciones establecen las relaciones que toda asignación aceptada debe cumplir. Si la entrada es distinta de cero, la segunda restricción obliga a que la salida sea nula, y la primera obliga entonces a que `inv` sea el inverso de la entrada. Si la entrada es nula, la primera restricción obliga a que la salida sea 1, sin determinar un único valor de `inv`. La garantía no consiste, por lo tanto, en que cualquier valor de `inv` satisfaga las restricciones, sino en que toda asignación que las satisface produce la salida correcta. El valor calculado sin restricción actúa como una sugerencia que las restricciones verifican. Si, en cambio, un circuito asignara su salida mediante `y <-- w * w;` sin ninguna restricción adicional, el compilador no generaría ninguna ecuación que vincule y con w, y un probador deshonesto podría generar una prueba válida para cualquier valor de y. En este caso concreto, las comprobaciones descritas en la Sección 3.2.3 emiten advertencias, tanto por el uso de `<--` con una expresión cuadrática como por la presencia de señales que no aparecen en ninguna restricción [38]. Estas comprobaciones identifican patrones conocidos, pero no detectan todas las vulnerabilidades posibles ni sustituyen una revisión de seguridad del circuito, ya que determinar en general si un circuito está correctamente restringido requiere análisis específicos como los que se describen a continuación.

Los circuitos insuficientemente restringidos constituyen una de las categorías de vulnerabilidades más estudiadas en sistemas de prueba sucintos [21], y han motivado el desarrollo

de herramientas de análisis específicas. Pailoor et al. propusieron un método automático para detectar circuitos insuficientemente restringidos [41], y Wen et al. desarrollaron un marco de análisis estático para circuitos de Circom que, evaluado sobre más de 258 circuitos de proyectos de código abierto, detectó vulnerabilidades previamente desconocidas [42]. Conviene subrayar que este riesgo no es exclusivo de Circom, sino propio de todo enfoque en el que el desarrollador define explícitamente las restricciones. Circom lo hace visible porque expone de manera directa la diferencia entre asignar un valor y restringirlo.

Una segunda limitación, también común al enfoque basado en circuitos, es el costo de las operaciones que no son nativas del cuerpo finito, señalado en la Sección 3.1.4. La plantilla `Num2Bits` de circomlib, que descompone un valor en n bits, introduce una restricción de la forma $b_i(b_i - 1) = 0$ para cada bit y una restricción adicional que recompone el valor original [39]. Las funciones de hash que operan sobre palabras binarias, como SHA-256 y Keccak-256, deben construirse sobre descomposiciones de este tipo, lo que multiplica el número de restricciones. El formato R1CS, además, no incorpora de forma nativa los argumentos de búsqueda mencionados en la Sección 3.1.4, que se retoman en la Sección 3.3.

Estas limitaciones adquieren especial relevancia en las verificaciones propias de los puentes entre cadenas, en las que conviene distinguir tres afirmaciones diferentes. La primera es que una transacción fue incluida en un bloque, lo que se demuestra mediante una prueba de inclusión respecto de la raíz de transacciones del encabezado y exige recalcular hashes Keccak-256 sobre los nodos del árbol de Merkle-Patricia y sobre el propio encabezado. La segunda es que dicho bloque es auténtico y está finalizado según el consenso de la cadena de origen. La tercera es que la operación de bloqueo se ejecutó con éxito, ya que una transacción incluida en un bloque puede haber revertido su ejecución. La prueba de inclusión no establece por sí misma esta última propiedad, cuya demostración puede requerir evidencia adicional, como el recibo de la transacción, que registra su estado de ejecución y los eventos emitidos, junto con su inclusión respecto de la raíz de recibos del encabezado [43, 44]. La segunda afirmación, por su parte, no se reduce a la verificación de una firma. El protocolo de cliente ligero de Ethereum exige, además de verificar la firma agregada del comité de sincronización, comprobar el número de miembros participantes, el dominio de firma correspondiente a la versión del protocolo, la pertenencia del encabezado finalizado al estado certificado mediante una rama de Merkle y la continuidad del propio comité entre periodos [45]. A partir de la actualización Capella, el protocolo incorpora además la vinculación entre el encabezado de consenso y el encabezado de ejecución que contiene las raíces mencionadas [46]. Entre estas comprobaciones, la verificación de la firma es la más costosa de expresar como circuito. Esta firma emplea la curva BLS12-381, cuyo cuerpo base tiene 381 bits y excede, por lo tanto, al cuerpo de 254 bits sobre el que opera el circuito, de manera que cada elemento debe representarse mediante varios valores del cuerpo y cada operación aritmética debe emularse. La implementación de referencia circom-pairing, desarrollada en el marco de un grupo de trabajo de 0xPARC, documenta que la verificación de una única firma BLS requiere aproximadamente 19,2 millones de restricciones no lineales, aproximadamente 3,2 horas de compilación y una clave de prueba de 12 GB en un servidor de

32 núcleos y 256 GB de memoria [47]. Estas cifras, obtenidas sobre un hardware concreto, no constituyen una medición del presente trabajo, pero ilustran el orden de magnitud del costo que implica expresar verificaciones de consenso como circuitos aritméticos. La forma en que el puente desarrollado en este trabajo aborda cada una de las tres afirmaciones se expone en el Capítulo 4.

Circom ofrece, en definitiva, un control preciso sobre las restricciones generadas, lo que permite optimizar manualmente los circuitos y conocer con exactitud la relación que se probará. Ese control tiene como contrapartida una mayor dificultad de desarrollo, ya que el programador debe razonar simultáneamente sobre el cálculo del testigo y sobre las ecuaciones que lo restringen, y asume la responsabilidad de que ambos coincidan. Noir, analizado en la Sección 3.3, conserva el enfoque basado en circuitos, pero eleva el nivel de abstracción, de modo que el desarrollador describe la computación con construcciones más cercanas a las de un lenguaje de programación convencional y el compilador asume la generación de las restricciones correspondientes.

3.3. Noir

La Sección 3.2 concluyó que Circom ofrece un control preciso sobre las restricciones a costa de exigir al desarrollador que razone simultáneamente sobre el cálculo del testigo y sobre las ecuaciones que lo restringen. Noir conserva el enfoque basado en circuitos descrito en la Sección 3.1.4, pero eleva el nivel de abstracción, ya que el desarrollador describe la computación con construcciones próximas a las de un lenguaje de programación convencional y el compilador asume la generación de las restricciones. Este cambio desplaza el trabajo de traducción hacia el compilador, pero no la responsabilidad de especificar correctamente la relación que se pretende demostrar. La presente sección describe el origen y los componentes de Noir, la forma de describir computaciones en el lenguaje, el proceso que conduce desde el código fuente hasta una prueba verificable en Ethereum mediante el backend Barretenberg y las limitaciones relevantes para un puente entre cadenas. Dado que varios de estos aspectos cambian entre versiones, la descripción corresponde a las utilizadas en el proyecto, Nargo 1.0.0-beta.22 y Barretenberg 5.0.0-nightly.20260522.

3.3.1. Origen y propósito

Noir es un lenguaje de dominio específico, de código abierto, para escribir programas cuya ejecución correcta puede demostrarse mediante pruebas de conocimiento cero, desarrollado en el ecosistema de Aztec [3, 48]. Su documentación lo describe como un lenguaje influido por Rust [49], y su sintaxis adopta efectivamente construcciones de ese lenguaje, como la declaración de funciones con `fn`, las ligaduras con `let` o el sistema de módulos. Noir no es, sin embargo, un subconjunto de Rust. Posee un sistema de tipos propio, organizado alrededor del tipo `Field`, impone restricciones que no existen en Rust, como el tamaño fijo de las entradas del programa principal, y no es compatible con las bibliotecas de ese ecosistema.

En la cadena de herramientas de Noir conviene distinguir cinco elementos con funciones diferentes. El primero es el lenguaje y su compilador, que traduce el programa a una representación intermedia denominada ACIR, descrita en la Sección 3.3.3. El segundo es Nargo, la herramienta de línea de órdenes que gestiona los proyectos y sus dependencias e invoca al compilador, al ejecutor y a las pruebas unitarias. El tercero corresponde a los componentes que calculan el testigo ejecutando la representación intermedia, disponibles tanto a través de Nargo como de la biblioteca `noir_js`, que permite hacerlo desde JavaScript [50]. El cuarto es el backend de pruebas, en este caso Barretenberg, una biblioteca en C++ que se utiliza mediante la interfaz de línea de órdenes `bb` o mediante `bb.js`, su envoltorio para JavaScript y WebAssembly [51, 52]. El quinto es el sistema criptográfico que implementa ese backend, UltraHonk, cuyas características se describen en la Sección 3.3.3.

Esta separación sustenta la independencia de Noir respecto del backend, ya que el repositorio oficial declara que el lenguaje fue diseñado para utilizar cualquier sistema de prueba compatible con ACIR [48]. Dicha independencia tiene, no obstante, condiciones precisas. El backend debe interpretar todos los tipos de instrucciones de ACIR, incluidas las funciones de caja negra que la especificación delega en él [49]. El tamaño del tipo `Field` depende del cuerpo nativo del backend, por lo que un programa cuya corrección dependa del módulo no es trasladable sin revisión [49]. Además, los formatos de los artefactos y de las pruebas están ligados a versiones concretas de las herramientas, motivo por el cual el proyecto exige utilizar pares de versiones de Nargo y Barretenberg que coincidan entre sí. La independencia se sitúa, por lo tanto, en el nivel del lenguaje y de la representación intermedia, y no implica que cualquier versión de cualquier herramienta pueda intercambiarse.

El interés de Noir para el presente trabajo reside en la forma de representar la computación. La verificación de una prueba de inclusión sobre un árbol de Merkle-Patricia exige decodificar estructuras RLP, recorrer nodos y recalcular hashes Keccak-256, operaciones que en Noir pueden expresarse con funciones, ciclos acotados y arreglos, sin escribir manualmente cada restricción. Al mismo tiempo, Noir permanece dentro del enfoque basado en circuitos, de modo que el sistema resultante describe específicamente esa verificación, y Barretenberg permite generar verificadores en Solidity para comprobar las pruebas en Ethereum. Las consecuencias de estas características sobre el rendimiento no se anticipan en este capítulo, sino que forman parte de la comparación experimental del Capítulo 4.

3.3.2. Descripción de computaciones y generación de restricciones

Un programa Noir se compone de funciones cuyos parámetros tienen tipos conocidos durante la compilación. Dentro de ellas, las ligaduras declaradas con `let` son inmutables salvo que se indique `mut`, las operaciones aritméticas se escriben con la notación habitual y la instrucción `assert` establece una condición que toda ejecución válida debe satisfacer. La función `main` es el punto de entrada del programa y sus parámetros deben tener un tamaño fijo, conocido durante la compilación [49].

El tipo `Field` representa un elemento del cuerpo nativo del backend, que en el caso de Barretenberg es un cuerpo primo de 254 bits [49]. Su aritmética es la de $\mathbb{F}_p$ descrita en la Sección 3.1.4, por lo que el resultado se reduce módulo p sin ninguna comprobación de desbordamiento. Los tipos enteros, como `u8`, `u32` o `u64`, son en cambio valores del cuerpo restringidos a un rango [49]. En una compilación realizada para el presente trabajo con Nargo 1.0.0-beta.22, una función que suma dos parámetros `u8` dio lugar a restricciones de rango de 8 bits sobre cada parámetro y sobre el resultado de la suma, de modo que la ejecución con los valores 200 y 100 falló por desbordamiento, mientras que la suma de $p-1$ y 1 con el tipo `Field` produjo 0 sin error. En esta operación concreta, la suma sobre `Field` evita las restricciones de rango y la comprobación de desbordamiento que requiere la suma de enteros, aunque exige al desarrollador garantizar que el comportamiento modular sea el previsto. El ejemplo no permite, sin embargo, concluir que `Field` sea más económico en cualquier programa, ya que el costo final depende también de las operaciones posteriores, de las comprobaciones que la aplicación necesite y de las optimizaciones del backend. Los enteros, por su parte, incorporan automáticamente las restricciones necesarias para reproducir la aritmética de ancho fijo.

La visibilidad se declara de forma explícita en la firma de la función principal. Sus parámetros son privados salvo que se marquen con `pub`, en cuyo caso forman parte de las entradas públicas, y su valor de retorno, si existe, debe declararse obligatoriamente como público, ya que el compilador rechaza un tipo de retorno de `main` sin `pub`. A diferencia de Circom, donde las salidas del componente principal son siempre públicas y las entradas se hacen públicas mediante una lista en la declaración de `main` (Sección 3.2.2), en Noir la visibilidad se indica individualmente para cada parámetro.

La Figura 3.5 muestra un programa que expresa la misma relación que el circuito de la Figura 3.2, $y = w^3 + w + 5$, con w privado e y público. Se eligió una interfaz con una entrada pública comprobada mediante una aserción, en lugar de un valor de retorno público, porque reproduce el modo en que un contrato verificador recibe el valor público y lo contrasta con sus propios datos. Ambas formulaciones son válidas y en las dos el valor de y forma parte del enunciado.

```
1 // w: privado; y: publico
2 fn main(w: Field, y: pub Field) {
3     assert(w * w * w + w + 5 == y);
4 }
```

Figura 3.5: Programa Noir que expresa la relación $w^3 + w + 5 = y$.

En este programa, el probador aporta el valor privado w y el valor público y, mientras que el enunciado contiene únicamente y. La prueba respalda que el probador conoce un w tal que $w^3 + w + 5 = y$ en $\mathbb{F}_p$, relación que, al tratarse del tipo `Field`, se evalúa módulo p. El desarrollador no declaró valores intermedios, ya que es el compilador quien traduce la expresión a la representación intermedia. En la compilación realizada para el presente trabajo, el programa dio lugar a dos aserciones aritméticas, $w_2 = w_0 \cdot w_0$ y $w_1 = w_0 \cdot w_2 + w_0 + 5$, donde

w_0 corresponde a w, w_1 a y y w_2 es un valor intermedio introducido por el compilador, análogo a la señal v_1 del ejemplo de Circom. La ejecución con $w = 3$ e $y = 35$ produjo un testigo válido, mientras que con $y = 36$ falló porque la aserción no puede satisfacerse. La relación demostrada es matemáticamente equivalente a la del ejemplo de Circom, pero su representación no lo es. Noir no produce un sistema R1CS, sino la representación intermedia descrita a continuación, que el backend transforma a su vez en las restricciones de su propio sistema de prueba, por lo que el número de restricciones y el costo de ambos ejemplos no son comparables directamente.

3.3.3. Compilación, cálculo del testigo y generación de la prueba

El compilador de Noir produce un programa en **ACIR** (*Abstract Circuit Intermediate Representation*), una representación intermedia diseñada para servir de enlace entre un lenguaje de alto nivel y un sistema de prueba, independiente de cualquier sistema concreto [48]. Un programa ACIR es una secuencia de instrucciones, denominadas *opcodes*, que restringen un conjunto de valores llamados testigos. La instrucción más básica es una aserción sobre una expresión, entendida como la suma de términos lineales en los testigos, de términos formados por el producto de dos testigos y de un término constante. Una fila de R1CS relaciona el producto de dos combinaciones lineales con una tercera (Sección 3.2.2), mientras que una expresión de ACIR puede contener varios términos de producto independientes. La restricción $e = a \cdot b + c \cdot d$, que no puede escribirse como el producto de dos combinaciones lineales y que el compilador de Circom rechaza, se compiló en Noir como una única aserción. ACIR incluye además llamadas a funciones de caja negra, cuya implementación se delega en el backend, operaciones sobre memoria y llamadas a código no restringido [48, 49]. Los testigos de ACIR se denominan *parciales*, porque el backend puede introducir testigos y restricciones adicionales al construir su propio circuito, en particular al expandir las funciones de caja negra.

La representación intermedia debe distinguirse de su ejecución. La **ACVM** (*Abstract Circuit Virtual Machine*) es la máquina virtual que ejecuta un programa ACIR sobre las entradas proporcionadas por el usuario y obtiene el testigo parcial que el backend necesita para generar la prueba [48]. **Brillig**, por su parte, es un formato de código y una máquina virtual de propósito general destinados a ejecutar las funciones no restringidas de Noir. Cuando el programa ACIR contiene una llamada a código no restringido, la ACVM delega su ejecución en la máquina virtual de Brillig e incorpora los resultados al testigo [48]. Ni ACIR, ni la ACVM, ni Brillig constituyen un sistema criptográfico, ya que la generación y verificación de la prueba corresponde exclusivamente al backend.

La Figura 3.6 representa el recorrido completo desde el código fuente hasta una prueba verificable, tal como se organiza con las herramientas utilizadas en el proyecto.

La primera etapa es la compilación. Nargo compila el programa y produce un artefacto que contiene el código ACIR, el código Brillig de las funciones no restringidas y una descripción de la interfaz del programa principal, que indica el tipo y la visibilidad de cada entrada. La segunda etapa consiste en preparar las entradas concretas, ya sea en el archivo de configuración

que utiliza Nargo o como un objeto de JavaScript cuando se emplea `noir_js`, codificadas según esa interfaz. En la tercera etapa, la ACVM ejecuta el programa sobre esas entradas y obtiene el testigo, ya sea mediante la orden `nargo execute` o mediante la función de ejecución de `noir_js` [49, 50]. Si alguna aserción no puede satisfacerse, la ejecución falla y no se obtiene ningún testigo.

La cuarta etapa corresponde a la preparación del backend. Barretenberg construye su propio circuito a partir del código ACIR, expandiendo las funciones de caja negra, y necesita una cadena de referencia común cuyo tamaño depende del tamaño del circuito. La clave de verificación se deriva del código ACIR y de esa cadena sin necesitar el testigo de una ejecución válida del programa. En la versión utilizada, `bb` construye el circuito para ese fin con valores auxiliares generados rápidamente y no válidos, que solo sirven para expandir las funciones de caja negra, de modo que la clave puede generarse una única vez por circuito, antes de cualquier operación concreta, y utilizarse para producir el contrato verificador en Solidity [51]. Esta etapa no depende, por lo tanto, de la tercera, y ambas pueden realizarse de forma independiente. En la quinta etapa, el backend combina el código ACIR, el testigo y la cadena de referencia para generar la prueba π junto con los valores de las entradas públicas, mediante la orden `bb prove` o mediante la clase `UltraHonkBackend` de `bb.js` [51, 52]. Finalmente, la verificación puede realizarse con `bb`, con `bb.js` o mediante el contrato generado, que requiere los contratos precompilados de Ethereum para la suma, la multiplicación escalar y el emparejamiento sobre la curva BN254, así como el de exponenciación modular [51].

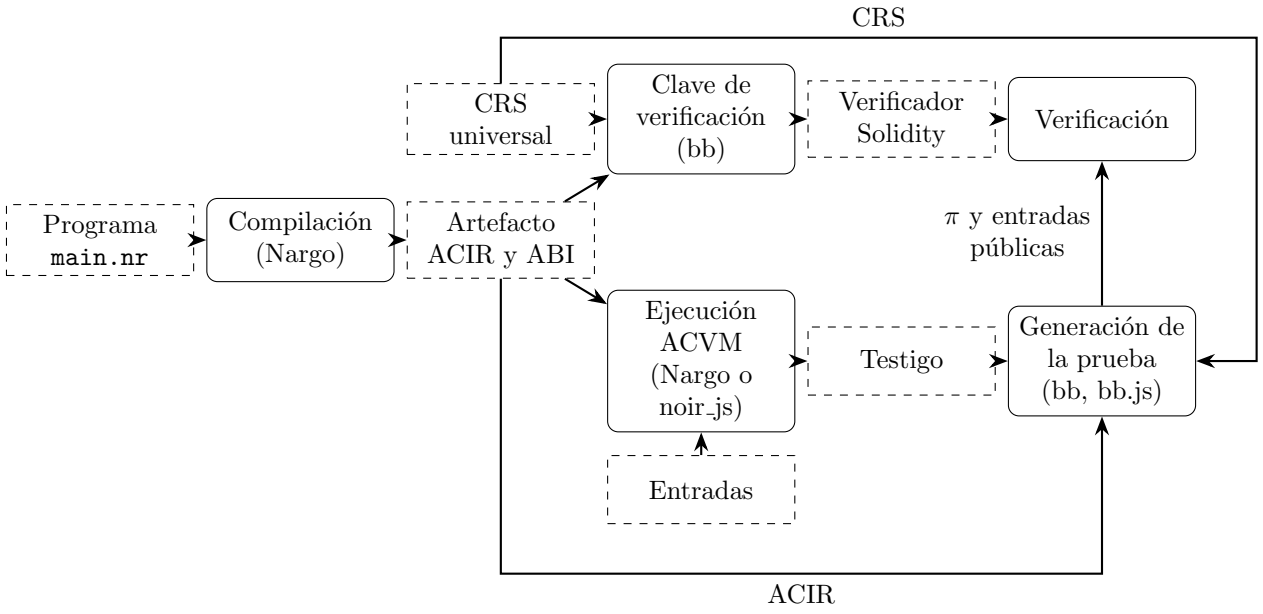


Figura 3.6: Proceso de compilación, cálculo del testigo, generación y verificación de una prueba a partir de un programa Noir con el backend Barretenberg.

Según su documentación y su código fuente, el prototipo del puente distribuye estas herramientas en dos momentos. Nargo y la interfaz `bb` se emplean durante la preparación, para compilar el circuito y generar la clave de verificación y el contrato verificador. Durante cada operación del puente, en cambio, el servicio que genera las pruebas utiliza `noir_js` para calcular

el testigo y `bb.js` para generar la prueba, mientras que la verificación se realiza en la cadena de destino mediante el contrato generado. Los detalles de esta integración se describen en el Capítulo 4.

El sistema de prueba que implementa Barretenberg en esta modalidad es **UltraHonk**, al que su documentación describe como un sistema de tipo PLONK más eficiente que su predecesor UltraPlonk [31, 51]. Su aritmetización organiza el circuito como una tabla de ejecución con un número fijo de columnas, en la que cada fila representa una compuerta cuyo tipo se determina mediante polinomios selectores. Además de las compuertas aritméticas, la clave de verificación generada en la versión utilizada declara selectores para compuertas especializadas, entre ellas las de comprobación de rango, operaciones sobre curvas elípticas, accesos a memoria, aritmética sobre cuerpos no nativos y la función de hash Poseidon2. La igualdad entre valores de distintas filas se impone mediante restricciones de copia, como en PLONK [30], y la pertenencia de ciertos valores a tablas precomputadas se comprueba mediante un argumento de búsqueda. Esta técnica fue introducida en este contexto por Plookup [53], aunque la versión de Barretenberg utilizada implementa una variante basada en derivadas logarítmicas [54, 51]. Estas **tablas de búsqueda** permiten verificar operaciones como las comprobaciones de rango o las operaciones bit a bit consultando una tabla, en lugar de descomponer cada valor en bits mediante restricciones aritméticas, como se describió para Circom en la Sección 3.2.4. Las relaciones del circuito se verifican mediante el protocolo *sumcheck*, que la documentación de Barretenberg incluye entre sus componentes con y sin conocimiento cero [51]. Para el ejemplo de la Figura 3.5, la herramienta `bb gates` informó un circuito de 20 filas, cifra que incluye filas fijas del sistema y que ilustra que el número de instrucciones ACIR no coincide con el tamaño del circuito que finalmente se prueba.

Tres propiedades de esta modalidad deben precisarse, porque no se aplican automáticamente a cualquier ejecución de Noir. En primer lugar, la variante destinada a Ethereum emplea compromisos KZG sobre la curva BN254 [51], por lo que requiere una configuración confiable de tipo universal, en el sentido de la Sección 3.1.5. Barretenberg obtiene la cadena de referencia de la ceremonia Ignition de Aztec, en la que participaron 176 personas e instituciones entre 2019 y 2020 y cuya seguridad requiere que al menos un participante haya destruido su aleatoriedad [55]. Los puntos necesarios se descargan cuando no están disponibles localmente y su integridad se comprueba mediante hashes [51]. A diferencia del flujo de Groth16 descrito para Circom, no se requiere una fase específica para cada circuito. En segundo lugar, la propiedad de conocimiento cero depende de la variante elegida. El destino `evm` de `bb` genera pruebas con conocimiento cero y utiliza Keccak como función de hash en la transformación de Fiat-Shamir, mientras que el destino `evm-no-zk` prescinde de dicha propiedad [51]. En tercer lugar, la verificación en Ethereum exige que la prueba, la clave de verificación y el contrato verificador se generen con el mismo destino. En una ejecución realizada para el presente trabajo con Barretenberg 5.0.0-nightly.20260522, la prueba del ejemplo con el destino `evm` ocupó 4544 bytes, un tamaño considerablemente mayor que el de una prueba Groth16, y fue verificada correctamente con `bb` y con `bb.js`.

Esta descripción permite identificar intervalos de tiempo de naturaleza distinta. La compilación, la generación de la clave de verificación y del contrato verificador y el despliegue de este último constituyen una preparación que se realiza una vez por circuito. La descarga de la cadena de referencia y la inicialización del backend son costos de puesta en marcha que pueden presentarse solo en la primera operación. El cálculo del testigo, la generación de la prueba y su eventual verificación local se repiten en cada operación, mientras que la verificación en la cadena de destino forma parte de una transacción, cuyo costo se expresa en gas y cuya duración incluye la espera hasta su inclusión en un bloque. Estas etapas no tienen necesariamente un equivalente directo en SP1, por lo que su correspondencia se establece en la metodología del Capítulo 4.

3.3.4. Limitaciones y consideraciones de seguridad

La primera limitación se refiere a la diferencia entre elevar el nivel de abstracción y garantizar la corrección de la especificación. En el código restringido, el compilador genera las restricciones correspondientes a cada instrucción, de modo que el desarrollador no necesita escribir por separado el cálculo y la restricción de cada valor, como ocurre en Circom. Esta automatización no elimina, sin embargo, todos los riesgos de restricciones insuficientes. Los resultados del cálculo no restringido, descrito más adelante, requieren comprobaciones explícitas cuando intervienen en la relación demostrada, y la prueba sigue certificando únicamente lo que el programa expresa. Un programa que verifica la inclusión de una transacción respecto de una raíz suministrada como entrada establece solo esa inclusión, y no la autenticidad de la raíz, la finalidad del bloque ni el éxito de la operación, distinciones ya presentadas en la Sección 3.2.4. Del mismo modo, una condición omitida en el programa no produce un error de compilación, sino una relación más débil que la prevista. Los errores de especificación de este tipo forman parte de las vulnerabilidades documentadas en sistemas basados en pruebas sucintas [21], y el propio repositorio de Noir advierte que la implementación no ha sido auditada ni revisada para su uso en producción [48].

```
1  unconstrained fn inverse(x: Field) -> Field {
2      if x == 0 { 0 } else { 1 / x }
3  }
4
5  fn main(x: Field) -> pub Field {
6      // Safety: el resultado se restringe a continuacion
7      let inv = unsafe { inverse(x) };
8      assert(x * inv == 1);
9      inv
10 }
```

Figura 3.7: Cálculo no restringido de un inverso en Noir, seguido de la restricción que lo valida.

Noir conserva, además, una forma de cálculo no restringido. Las funciones declaradas como `unconstrained` se compilan a código Brillig y se ejecutan fuera del sistema de restricciones, de

manera que sus resultados se incorporan al testigo sin que ninguna restricción los respalde [49]. La documentación justifica este mecanismo porque ciertas operaciones resultan mucho más baratas de comprobar que de calcular dentro del circuito, y exige por ello que el programa añada las restricciones necesarias sobre sus resultados. La Figura 3.7 muestra este patrón para el cálculo de un inverso.

El ejemplo presenta dos particularidades que conviene precisar. Cuando $x = 0$, la función auxiliar devuelve cero, pero la aserción $x \cdot inv = 1$ no puede satisfacerse, de modo que el programa completo solo acepta entradas no nulas. Además, el inverso se devuelve como valor público, y para una entrada no nula basta con invertirlo para recuperar x, por lo que la declaración de x como privada no mantiene su valor en secreto. Esto no contradice la propiedad de conocimiento cero (Sección 3.1.2), que garantiza que la prueba no revela información sobre el testigo más allá de la validez del enunciado, pero no oculta lo que puede deducirse de los propios valores públicos.

La semejanza con el operador `<--` de Circom se limita a que, en ambos casos, un valor entra en el testigo sin quedar restringido por la operación que lo calcula. Las diferencias son relevantes. Una función no restringida de Noir puede contener una computación completa, con ciclos y bifurcaciones, ejecutada por la máquina virtual de Brillig, y no una única asignación. Además, el lenguaje exige que su invocación desde código restringido se realice dentro de un bloque `unsafe`, cuya ausencia produce un error de compilación, y la versión utilizada advierte cuando ese bloque carece de un comentario que justifique su seguridad. En una compilación realizada para el presente trabajo, la eliminación de la aserción de la Figura 3.7 produjo un diagnóstico que indicaba que los resultados de la llamada no estaban suficientemente restringidos, aunque el programa se compiló y ejecutó de todos modos. Este diagnóstico, como las comprobaciones de Circom, identifica patrones conocidos, pero no sustituye una revisión del programa.

La vinculación entre los datos públicos y la relación demostrada constituye una segunda consideración. Como se señaló en la Sección 3.1.4, la garantía de la prueba alcanza únicamente a los valores que forman parte del enunciado, de modo que, si un contrato acuña activos a favor de un destinatario y por un monto determinados, ambos valores deben formar parte de las entradas públicas o estar vinculados criptográficamente a un valor público que el contrato compruebe. En el primer caso, el contrato compara directamente las entradas públicas con los parámetros de su propia llamada. En el segundo, el enunciado puede contener, por ejemplo, el hash de una codificación inequívoca de esos datos, de modo que el circuito restringe la relación entre el hash y los valores, y el contrato calcula el mismo hash a partir de los parámetros de su llamada y lo compara con la entrada pública. La interfaz concreta del prototipo se describe en el Capítulo 4. En cualquier caso, la codificación de las entradas públicas debe conocerse con precisión. En la versión utilizada, por ejemplo, un arreglo público de n elementos `u8` se expone como n entradas públicas, cada una restringida a 8 bits, por lo que el contrato debe reconstruir los valores a partir de esa representación.

Las restricciones sobre el control de flujo y el tamaño de los datos derivan del enfoque basado en circuitos. En el código restringido, el número de iteraciones de cada ciclo debe conocerse

durante la compilación, y las instrucciones para interrumpir un ciclo solo se admiten en código no restringido [49]. Los parámetros del programa principal deben tener un tamaño fijo, por lo que datos de longitud variable, como una transacción codificada en RLP o los nodos de una prueba de Merkle-Patricia, deben representarse mediante arreglos de un tamaño máximo acompañados de su longitud efectiva. El circuito incluye, en consecuencia, las restricciones correspondientes al tamaño máximo con independencia de la longitud real de los datos.

El costo de los hashes binarios y de la aritmética sobre cuerpos no nativos tampoco desaparece con el cambio de lenguaje. Noir ofrece algunas de estas operaciones como funciones de caja negra, entre ellas la permutación de Keccak, la compresión de SHA-256 y las operaciones bit a bit, cuya implementación se delega en el backend para que este utilice restricciones optimizadas cuando dispone de ellas [49]. La función Keccak-256 completa, en cambio, no forma parte de la biblioteca estándar de la versión utilizada y debe construirse sobre esa permutación o tomarse de bibliotecas externas [49]. Las tablas de búsqueda reducen el costo de las comprobaciones de rango y de las operaciones bit a bit, pero no eliminan el costo de emular la aritmética de un cuerpo de mayor tamaño, como el de 381 bits de la curva BLS12-381 que emplea la firma del comité de sincronización de Ethereum, cuyos elementos deben seguir representándose mediante varios valores del cuerpo nativo.

Por último, los costos de un programa Noir dependen del backend, de sus parámetros y de las versiones utilizadas. El número de instrucciones ACIR no determina por sí solo el tamaño del circuito, que depende de la forma en que el backend expande cada instrucción, y ese tamaño condiciona a su vez la cantidad de puntos de la cadena de referencia y la memoria necesarios para generar la prueba. Un cambio de versión de Nargo o de Barretenberg puede modificar la representación intermedia, el circuito resultante o el formato de las pruebas, por lo que cualquier medición debe asociarse a las versiones con que se obtuvo.

Noir permite describir la verificación como un programa, pero cada programa sigue dando lugar a un circuito específico. La Sección 3.4 estudia el enfoque alternativo presentado en la Sección 3.1.4, en el que se demuestra la ejecución de un programa arbitrario sobre una máquina virtual de conocimiento cero.

3.4. SP1

La Sección 3.3 concluyó que Noir permite describir la verificación como un programa, pero que cada programa sigue dando lugar a un circuito específico. SP1 adopta el enfoque alternativo presentado en la Sección 3.1.4, en el que el sistema de restricciones describe un procesador y la prueba certifica que un programa arbitrario se ejecutó correctamente sobre él. Para el desarrollador, el cambio consiste en escribir un programa convencional en Rust y delegar en la máquina virtual la representación de su ejecución. Este cambio no elimina la necesidad de especificar correctamente la relación que se pretende demostrar, sino que la traslada al programa que se ejecuta. La presente sección describe el origen y los componentes de SP1, la forma de escribir programas y de definir sus valores públicos, el proceso que conduce desde el código

fuente hasta una prueba verificable en Ethereum y las limitaciones relevantes para un puente entre cadenas. La descripción corresponde a la serie 6 de SP1, en particular a las versiones 6.1.0 y 6.2.2 que resuelven las dependencias del proyecto, ya que esta serie introdujo cambios de arquitectura respecto de las anteriores.

3.4.1. Origen y propósito

SP1 es una máquina virtual de conocimiento cero de código abierto, desarrollada por Succinct Labs, que permite demostrar la ejecución correcta de programas compilados para la arquitectura RISC-V [56, 57]. Su repositorio reconoce como antecedentes directos el conjunto de herramientas criptográficas Plonky3, el diseño de tablas y de búsquedas cruzadas de la máquina virtual Valida, la cadena de compilación de RISC Zero y la idea de los *builtins* de Cairo, que SP1 generaliza mediante precompilados implementados como tablas adicionales [56].

La serie 6, publicada a partir de febrero de 2026, incorporó un sistema de pruebas denominado Hypercube, basado en polinomios multilineales [57]. En esta serie, la máquina virtual implementa el conjunto de instrucciones RV64IM, es decir, la base entera de 64 bits de RISC-V junto con la extensión M de multiplicación y división [58, 57], y los programas se compilan para el destino `riscv64im-succinct-zkvm-elf` [56]. El conjunto no incluye instrucciones de punto flotante, de modo que esas operaciones deben emularse mediante software. Las versiones anteriores empleaban un conjunto de instrucciones de 32 bits, y la documentación advierte que las cadenas de compilación de 32 y 64 bits son incompatibles entre sí [57]. Los valores con los que opera el sistema de pruebas pertenecen al cuerpo primo KoalaBear, de módulo $p = 2^{31} - 2^{24} + 1$, y a una extensión de grado 4 de dicho cuerpo [56].

Al igual que en el caso de Noir, conviene distinguir los componentes que intervienen. El programa se escribe en Rust, aunque el repositorio indica que puede emplearse cualquier lenguaje compilado mediante LLVM [56]. Su compilación al conjunto de instrucciones admitido se realiza con una cadena de compilación de Rust distribuida por Succinct, que produce un ejecutable en formato ELF. La máquina virtual ejecuta ese archivo y registra los datos necesarios para demostrar su ejecución. El sistema de pruebas, formado por el protocolo Hypercube, un mecanismo de recursión y una etapa final que traduce la prueba a Groth16 o PLONK, genera la prueba a partir de esos datos. Por último, la verificación puede realizarse con el kit de desarrollo de SP1, escrito en Rust, o mediante contratos verificadores en Solidity publicados por Succinct [56, 59].

La reutilización de código constituye el principal atractivo de este enfoque. El repositorio indica que los programas pueden utilizar la biblioteca estándar de Rust y que la mayoría de las bibliotecas de su ecosistema deberían ser compatibles [56]. Los programas del puente utilizan, por ejemplo, bibliotecas existentes para SHA-256, Keccak-256, aritmética sobre la curva BLS12-381 y codificación según el ABI de Solidity. Esta compatibilidad tiene, sin embargo, límites. El programa se ejecuta sin sistema operativo, por lo que solo dispone de las llamadas al sistema que define la propia máquina virtual, como la lectura de entradas, la publicación de valores, la

finalización de la ejecución y los precompilados descritos en la Sección 3.4.3 [56]. Las bibliotecas que dependen del acceso a archivos, a la red o a otros servicios del sistema operativo no pueden utilizarse sin adaptación, y toda dependencia debe poder compilarse para el destino de 64 bits mencionado. Que una biblioteca compile, además, no implica que su costo de prueba resulte razonable, cuestión que se retoma en la Sección 3.4.4.

SP1 debe distinguirse, asimismo, de la infraestructura de generación de pruebas que ofrece Succinct. El kit de desarrollo permite generar las pruebas localmente, en procesador o en GPU, y también delegarlas en una red de probadores remota, funcionalidad que requiere habilitar una característica específica del kit y credenciales del servicio [56]. Los manifiestos del proyecto no habilitan esa característica, por lo que su implementación genera las pruebas de forma local.

El interés de SP1 para el presente trabajo reside en que materializa el enfoque basado en la ejecución de programas que la hipótesis contrapone al de Noir. La lógica de verificación de un puente, que incluye la decodificación de estructuras RLP, el cálculo de hashes y la verificación de firmas, puede escribirse como un programa Rust convencional que reutiliza bibliotecas existentes, mientras que la máquina virtual asume su representación como sistema de restricciones. Las consecuencias de este enfoque sobre los tiempos de prueba no se anticipan en este capítulo, sino que forman parte de la comparación del Capítulo 4.

3.4.2. Programas, entradas y valores públicos

Una aplicación de SP1 se compone de dos programas. El **programa invitado** (*guest*) es el que se compila para la máquina virtual y cuya ejecución se demuestra. El **programa anfitrión** (*host*) es un programa Rust convencional que se ejecuta fuera de la máquina virtual y utiliza el kit de desarrollo para preparar las entradas, solicitar la ejecución o la generación de la prueba y procesar el resultado [56, 57]. La prueba respalda únicamente la ejecución del programa invitado. Las operaciones del anfitrión, como la obtención de datos de un nodo de la cadena de origen, su decodificación o la construcción de las entradas, permanecen fuera de la ejecución demostrada, por lo que, si la corrección de esos datos es relevante, el programa invitado debe comprobarla por sí mismo.

El anfitrión escribe las entradas en un objeto `SP1Stdin`, que las serializa, y el programa invitado las lee mediante la función `sp1_zkvm::io::read`, que las deserializa, o mediante `read_vec`, que devuelve los bytes sin procesar [56]. La documentación del código advierte que este flujo de entrada está controlado por el probador [56]. Las entradas cumplen, por lo tanto, el papel del testigo de la Sección 3.1.1, ya que nada garantiza que correspondan a datos reales salvo las comprobaciones que el propio programa realice sobre ellas.

Las condiciones se comprueban con las construcciones ordinarias de Rust, como la macro `assert!` o el tratamiento de errores que interrumpe la ejecución. Una comprobación fallida provoca la terminación del programa con un código de salida distinto de cero, y una ejecución que termina de ese modo no puede dar lugar a una prueba que los verificadores acepten. La función de verificación del kit comprueba por defecto que la ejecución haya terminado con éxito,

y el contrato verificador de la versión utilizada rechaza cualquier código de salida distinto de cero [56, 59].

Los valores públicos se publican mediante `sp1_zkvm::io::commit`, que serializa un valor, o mediante `commit_slice`, que añade bytes sin procesar [56]. El conjunto de bytes publicados forma parte del enunciado, se entrega al verificador junto con la prueba y, en el contrato verificador, interviene a través de su hash SHA-256 [59]. Su codificación debe acordarse con quien lo consume, de modo que un contrato en Solidity, por ejemplo, necesita una codificación que pueda decodificar sin ambigüedad, como la del ABI de Ethereum.

El programa cuya ejecución se demuestra se identifica mediante su clave de verificación, que el kit deriva de forma determinista del ejecutable ELF durante la etapa de preparación [56]. En el contrato verificador, esa clave se representa mediante un valor de 32 bytes que se proporciona junto con los valores públicos y la prueba [59]. Quien verifica debe fijar el valor esperado, ya que, si no lo comprobara, aceptaría también una prueba válida de otro programa que publicara valores con el mismo formato.

```
#![no_main]
sp1_zkvm::entrypoint!(main);

pub fn main() {
    // Entrada privada: la suministra el probador.
    let w = sp1_zkvm::io::read::<u64>();
    // Aritmetica comprobada: un desbordamiento detiene la ejecucion.
    let y = w
        .checked_mul(w)
        .and_then(|w2| w2.checked_mul(w))
        .and_then(|w3| w3.checked_add(w))
        .and_then(|s| s.checked_add(5))
        .expect("w fuera del dominio admitido");
    // Valor publico: pasa a formar parte del enunciado.
    sp1_zkvm::io::commit(&y);
}
```

Figura 3.8: Programa invitado de SP1 que calcula $y = w^3 + w + 5$ con aritmética entera comprobada.

Los datos que el programa lee y no publica no se entregan al verificador, pero esto no basta para afirmar que permanezcan secretos. Por una parte, los valores públicos pueden permitir deducirlos. Por otra, la propiedad de conocimiento cero depende del modo de prueba. Según el modelo de seguridad de SP1, las pruebas Groth16 y PLONK, generadas con la biblioteca gnark, son de conocimiento cero, mientras que las pruebas de núcleo y las comprimidas no satisfacen actualmente esa propiedad [60]. En concordancia con ello, el repositorio de la versión utilizada incluye un componente experimental y no auditado destinado a añadir conocimiento cero a los protocolos multilineales en los que se basan estas últimas [56]. En el diseño del puente, la prueba que se verifica en la cadena de destino es una prueba Groth16, aunque la propiedad de conocimiento cero no resulta necesaria en una aplicación cuyos datos son públicos

(Sección 3.1.1).

La Figura 3.8 muestra un programa invitado que retoma la relación $y = w^3 + w + 5$ de los ejemplos de Circom y Noir. A diferencia de estos, que evalúan la relación en $\mathbb{F}_p$, el programa emplea enteros de 64 bits sin signo, cuya aritmética no es la de un cuerpo finito. Para evitar cualquier ambigüedad, cada operación se realiza de forma comprobada, de modo que un resultado que exceda el rango de esos enteros interrumpe la ejecución en lugar de reducirse módulo 2^{64}. El programa acepta, por lo tanto, únicamente los valores de w para los que $w^3+w+5 < 2^{64}$, es decir, $0 \leq w \leq 2\,642\,245$. En ese dominio, el resultado coincide con el de los ejemplos anteriores, ya que el valor entero es menor que el módulo de 254 bits que estos emplean, pero fuera de él la relación no es equivalente, puesto que aquellos aceptan cualquier elemento del cuerpo.

El programa lee w como entrada privada, calcula y con aritmética comprobada y publica y. La Figura 3.9 muestra el código anfitrión imprescindible para este flujo, que escribe w en la entrada, deriva las claves a partir del ejecutable, genera y verifica una prueba y lee el valor público. La prueba respalda que el programa identificado por la clave de verificación, ejecutado con alguna entrada, terminó correctamente y publicó y. El verificador recibe la clave de verificación, los valores públicos y la prueba, pero no w. El ejemplo no ilustra, sin embargo, ninguna forma de confidencialidad, ya que la función es estrictamente creciente en el dominio admitido y el valor público y determina, por lo tanto, un único w que cualquiera puede calcular.

```
use sp1_sdk::{
    blocking::{ProveRequest, Prover, ProverClient},
    include_elf, Elf, ProvingKey, SP1Stdin,
};

const ELF: Elf = include_elf!("cubic-program");

fn main() {
    let client = ProverClient::from_env();
    let mut stdin = SP1Stdin::new();
    stdin.write(&3u64); // w: entrada privada

    let pk = client.setup(ELF).unwrap(); // claves derivadas del ELF
    let mut proof = client.prove(&pk, stdin).compressed().run().unwrap();
    client.verify(&proof, pk.verifying_key(), None).unwrap();

    let y: u64 = proof.public_values.read(); // valor publico
    println!("y = {y}");
}
```

Figura 3.9: Programa anfitrión que prepara la entrada, genera y verifica una prueba comprimida y lee el valor público.

En una compilación y ejecución realizadas para el presente trabajo con SP1 6.2.2, el ejecutable del programa invitado resultó ser un archivo ELF para RISC-V de 64 bits. La ejecución con $w = 3$ publicó $y = 35$ tras 4802 instrucciones, cifra que incluye la lectura y deserialización de la entrada, mientras que con $w = 3\,000\,000$ la comprobación de desbordamiento interrumpió

el programa, que terminó con código de salida 1. El programa anfitrión de la Figura 3.9 generó y verificó correctamente una prueba comprimida para $w = 3$.

3.4.3. Compilación, ejecución, generación y verificación de la prueba

La Figura 3.10 representa el recorrido desde el código fuente hasta una prueba verificable. La primera etapa es la compilación del programa invitado, que el kit realiza con la cadena de compilación de Succinct, ya sea desde el script de compilación del anfitrión o mediante una orden específica, y que produce un ejecutable ELF para RISC-V de 64 bits [56]. El anfitrión incorpora ese ejecutable en su propio binario. A partir de él, la etapa de preparación deriva de forma determinista la clave de prueba y la clave de verificación, sin emplear ningún valor secreto [56]. Esta preparación no debe confundirse con la configuración confiable de algunos SNARKs descrita en la Sección 3.1.5, ya que solo depende del programa. Las entradas, por su parte, se preparan en el anfitrión y se codifican al escribirlas en el objeto de entrada, con independencia de la preparación de las claves.

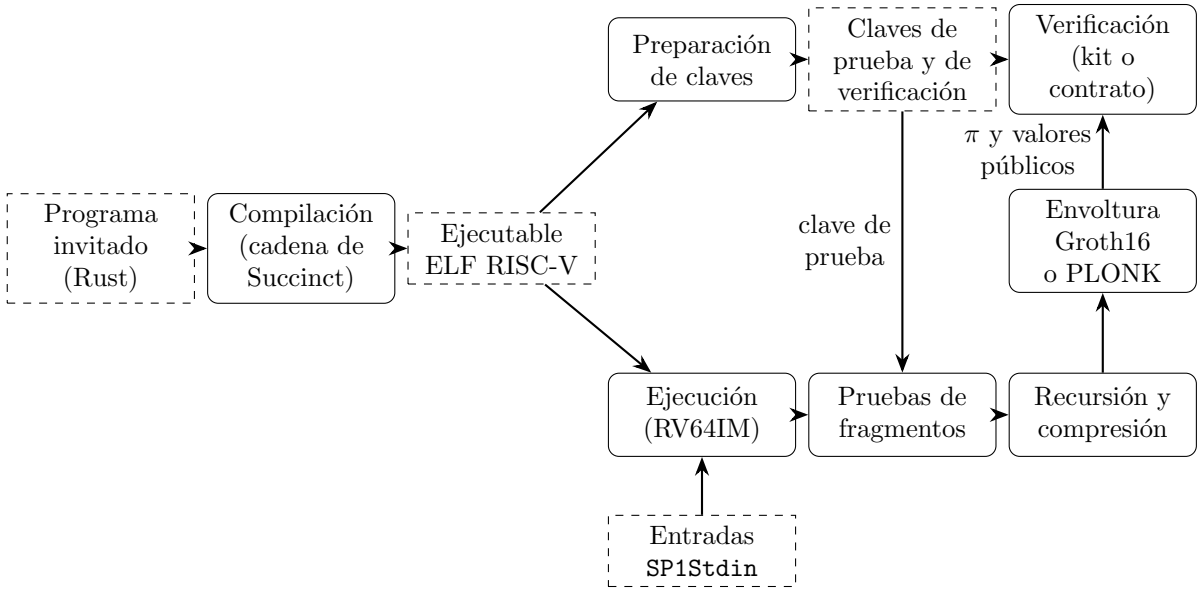


Figura 3.10: Proceso de compilación, ejecución, generación y verificación de una prueba con SP1.

En la etapa de ejecución, la máquina virtual ejecuta el programa sobre las entradas. El kit permite ejecutar sin generar ninguna prueba, en cuyo caso devuelve los valores públicos y un informe con el número de instrucciones ejecutadas y de llamadas al sistema [56]. Cuando se genera una prueba, la ejecución se divide además en **fragmentos** (*shards*), de aproximadamente 2^{22} instrucciones cada uno según la documentación [57], y los eventos registrados en cada fragmento dan lugar a la traza que se demuestra. La división acota los recursos necesarios para probar cada fragmento y permite procesarlos de forma independiente.

La traza de cada fragmento se organiza en tablas, en las que cada tabla corresponde a un tipo de instrucción o a un componente especializado y sus columnas contienen los operandos necesarios para comprobar la operación [57]. Dado que una palabra de 64 bits no cabe en

un elemento del cuerpo KoalaBear, cada palabra se representa mediante cuatro valores de 16 bits [56]. Las restricciones que relacionan las columnas de una misma tabla se comprueban mediante un protocolo de tipo *sumcheck*, denominado *zerocheck*, y las **interacciones** entre tablas, por ejemplo entre la tabla que registra una instrucción y las que verifican sus operandos, se comprueban mediante LogUp-GKR, un argumento de derivadas logarítmicas que verifica que los valores enviados y recibidos coinciden con sus multiplicidades [57, 61]. La coherencia de la memoria entre fragmentos se comprueba mediante un hash de multiconjuntos, que transforma cada lectura y escritura en un punto de una curva elíptica mediante la función Poseidon2, de modo que la igualdad entre los conjuntos de escrituras y de lecturas se reduce a la igualdad de sus hashes [57]. Por último, el probador se compromete con toda la traza del fragmento como un único polinomio multilineal, mediante un esquema de compromiso denominado *jagged*, que admite tablas de alturas distintas [62], construido sobre el esquema BaseFold [63, 56]. Este esquema se basa en funciones de hash, por lo que esta parte del sistema no requiere configuración confiable, y la interacción se elimina mediante la transformación de Fiat-Shamir (Sección 3.1.3).

Las pruebas de los fragmentos constituyen la prueba de núcleo, cuyo tamaño total es proporcional a la extensión de la ejecución [57]. Para obtener una prueba de tamaño constante, el mecanismo de recursión verifica las pruebas de los fragmentos dentro de la propia máquina de pruebas y las combina en forma de árbol hasta obtener una única prueba *comprimida* [57]. Esta prueba puede verificarse con el kit, pero no está pensada para su verificación en Ethereum. Para ello, una etapa adicional cambia la función de hash empleada por la recursión y, finalmente, una etapa de envoltura produce una prueba Groth16 o PLONK sobre la curva BN254 [57, 56]. Esta última etapa utiliza la biblioteca gnark [64] y puede ejecutarse mediante una imagen de Docker o, habilitando una característica del kit, de forma nativa [56]. Según la documentación, las claves del circuito de Groth16 proceden de la ceremonia Ignition, complementada con contribuciones del equipo de Succinct, mientras que la variante PLONK reutiliza la cadena de referencia universal de dicha ceremonia, y las pruebas resultantes ocupan unos 260 y 868 bytes, respectivamente [57].

La verificación puede realizarse con el kit, que admite las pruebas de cualquiera de los modos y comprueba por defecto que la ejecución terminó con éxito, o en Ethereum, con las pruebas Groth16 o PLONK [56, 57]. En este último caso, el contrato recibe la clave de verificación del programa, los valores públicos y la prueba. Los primeros cuatro bytes de la prueba identifican la versión del verificador, lo que permite a un contrato de enrutamiento dirigirla al verificador correspondiente. El contrato de Groth16 de la versión de circuito 6.1.0, que corresponde a las versiones utilizadas en el proyecto, comprueba la prueba con cinco entradas públicas, que son la clave del programa, el hash SHA-256 de los valores públicos truncado a 253 bits para que quepa en el cuerpo de BN254, el código de salida, que debe ser cero, una raíz que identifica las claves del mecanismo de recursión y un valor de uso único [59].

La máquina virtual ofrece, además, **precompilados**, es decir, tablas especializadas que demuestran operaciones costosas sin descomponerlas en instrucciones RISC-V. El programa los invoca mediante llamadas al sistema y, en la versión utilizada, abarcan la permutación de

Keccak, la expansión y la compresión de SHA-256, operaciones sobre las curvas secp256k1, secp256r1, ed25519, BN254 y BLS12-381, aritmética sobre los cuerpos de estas curvas y sobre enteros grandes, la función Poseidon2 y la verificación de pruebas de SP1 [56]. Su uso habitual no requiere escribir estas llamadas, sino sustituir bibliotecas criptográficas comunes por versiones modificadas que dirigen automáticamente sus operaciones hacia los precompilados [57]. Conviene distinguir, por lo tanto, el precompilado, que forma parte del sistema de pruebas, de la biblioteca modificada que lo invoca, y de las operaciones del anfitrión, que quedan fuera de la prueba. Si una dependencia no utiliza la versión modificada, la operación se ejecuta como instrucciones ordinarias y su costo aumenta considerablemente [57].

Esta descripción permite identificar intervalos de tiempo de naturaleza distinta. La compilación del programa invitado, la preparación de las claves y el despliegue del contrato verificador constituyen una preparación que se realiza una vez por programa. La construcción del cliente del kit y la obtención de los artefactos que necesita la etapa de envoltura son costos de puesta en marcha. En cada operación se repiten, en cambio, la preparación de las entradas en el anfitrión, que no forma parte de la prueba, la ejecución, la generación de las pruebas de los fragmentos, su compresión y, cuando se requiere la verificación en Ethereum, la envoltura final, seguidas de la verificación local y de la verificación en la cadena de destino, cuyo costo se expresa en gas y cuya duración incluye la espera hasta la inclusión de la transacción. El modo de ejecución simulado del kit, que produce valores públicos reales pero una prueba vacía, no genera ninguna prueba y no debe confundirse con estas etapas. Ninguna de ellas tiene necesariamente un equivalente directo en Noir, por lo que su correspondencia se establece en la metodología del Capítulo 4.

3.4.4. Limitaciones y consideraciones de seguridad

La primera limitación se refiere, como en Noir, a la diferencia entre la expresividad del lenguaje y la corrección de la especificación. La máquina virtual garantiza que el programa se ejecutó correctamente, pero no que el programa compruebe lo que se pretende. Dado que las entradas están controladas por el probador, un programa que reciba como entrada la raíz contra la que verifica la inclusión de una transacción, en lugar de derivarla de un encabezado cuya autenticidad también compruebe, demuestra únicamente la inclusión respecto de una raíz arbitraria. Las distinciones de la Sección 3.2.4 entre inclusión, autenticidad y finalidad del bloque y éxito de la operación siguen siendo responsabilidad del programa y de la aplicación que lo utiliza.

La segunda consideración es la identificación del programa. Toda modificación del código, de sus dependencias o de la cadena de compilación produce un ejecutable distinto y, en consecuencia, una clave de verificación distinta, de modo que el verificador debe fijar la clave correspondiente a una compilación concreta. Para que terceros puedan reproducir esa clave, la herramienta de compilación ofrece la opción de compilar dentro de un contenedor de Docker con el fin de obtener ejecutables reproducibles [56]. La verificación debe exigir, además, que la ejecución haya terminado con éxito, condición que el kit y el contrato verificador comprueban

por defecto.

SP1 admite también cálculo no restringido. El repositorio de la versión utilizada define una macro que ejecuta un bloque de código fuera de las restricciones, descarta al finalizar cualquier cambio en el estado de la máquina virtual y solo permite transmitir datos al programa a través de un flujo de sugerencias que este lee después como una entrada más [56]. Su papel es análogo al de las funciones no restringidas de Noir y al operador `<--` de Circom, ya que los datos así obtenidos solo quedan respaldados por las comprobaciones que el programa realice sobre ellos.

Los precompilados introducen supuestos propios. La documentación advierte que los precompilados de curvas elípticas asumen que sus entradas son puntos válidos de la curva, sin comprobarlo, que los valores de los cuerpos deben estar en su representación canónica y que ciertos casos, como el uso del precompilado de suma para duplicar un punto o la presencia del punto en el infinito, deben tratarse por separado para no producir pruebas imposibles de verificar [57]. Las bibliotecas modificadas deben corresponder, además, a la versión de SP1 utilizada, y una sustitución que no se aplique hace que la operación vuelva a ejecutarse como instrucciones ordinarias sin que se produzca ningún error [57].

El costo de la generalidad constituye la limitación central del enfoque. Cada instrucción ejecutada ocupa filas en las tablas de la traza, cada palabra de 64 bits se representa mediante varios elementos del cuerpo y las operaciones criptográficas que no utilizan precompilados se ejecutan como secuencias de instrucciones ordinarias. Los hashes binarios y la aritmética sobre la curva BLS12-381, presentes en la verificación de un puente, pueden dominar, en consecuencia, el número de instrucciones de un programa. El costo final depende, además, de la versión, de la configuración de los fragmentos, del hardware disponible, que incluye la posibilidad de emplear GPU, y del modo de prueba solicitado, ya que la envoltura en Groth16 o PLONK añade una etapa a la generación. Por ese motivo, cualquier medición debe asociarse a la versión, a la configuración y al hardware con que se obtuvo.

Por último, la privacidad de las entradas depende del modo de prueba y de quién la genera. Como se indicó en la Sección 3.4.2, solo las pruebas Groth16 y PLONK son de conocimiento cero, y esa propiedad protege las entradas frente al verificador, pero no frente a quien genera la prueba, que necesariamente las conoce. Por ese motivo, la delegación de la prueba en un servicio remoto implica, salvo en modalidades específicas que la documentación describe para ese fin, que el servicio recibe las entradas del programa [57]. En un puente cuyos datos son públicos esta limitación carece de consecuencias, pero debe tenerse en cuenta en otras aplicaciones.

Con SP1 concluye la exposición de los dos enfoques que el presente trabajo compara, el basado en circuitos específicos, representado por Noir, y el basado en la ejecución de programas sobre una máquina virtual, representado por SP1. La Sección 3.5 reseña brevemente otras herramientas del ecosistema, y el Capítulo 4 describe la implementación del puente con ambas tecnologías y la metodología con que se comparan.

3.5. Otros (Risc0, Succinct, Aligned, Cairo, Halo2, Plonky3)

CAPÍTULO 4

ZKBRIDGE

4.1. Implementación de un puente Lock-and-Mint

4.2. zkBridge de Berkeley RDI

4.3. Diseño e Implementación en Noir

4.4. Benchmarks

CAPÍTULO 5

CONCLUSIÓN

REFERENCIAS

[1] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. 2014. URL: https://ethereum.org/en/whitepaper/.

[2] Mengya Zhang, Xiaokuan Zhang, Josh Barbee, Yinqian Zhang y Zhiqiang Lin. *SoK: Security of Cross-chain Bridges: Attack Surfaces, Defenses, and Open Problems*. arXiv:2312.12573 [cs.CR]. 2023. URL: https://arxiv.org/abs/2312.12573.

[3] Aztec Labs. *Noir: A Domain Specific Language for Zero-Knowledge Proofs*. 2024. URL: https://noir-lang.org/docs/.

[4] Succinct Labs. *SP1: A Zero-Knowledge Virtual Machine for Rust*. 2024. URL: https://docs.succinct.xyz/docs/sp1/introduction.

[5] Ralph C. Merkle. «A Digital Signature Based on a Conventional Encryption Function». En: *Advances in Cryptology — CRYPTO '87* (1988), págs. 369-378. DOI: 10.1007/3-540-48184-2_32.

[6] Leslie Lamport, Robert Shostak y Marshall Pease. «The Byzantine Generals Problem». En: *ACM Transactions on Programming Languages and Systems* 4.3 (1982), págs. 382-401. URL: https://lamport.azurewebsites.net/pubs/byz.pdf.

[7] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. URL: https://bitcoin.org/bitcoin.pdf.

[8] Justin Thaler. «Proofs, Arguments, and Zero-Knowledge». En: *Foundations and Trends in Privacy and Security* 4.2–4 (2022), págs. 117-660. DOI: 10.1561/3300000030. URL: https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.pdf.

[9] Shafi Goldwasser, Silvio Micali y Charles Rackoff. «The Knowledge Complexity of Interactive Proof Systems». En: *SIAM Journal on Computing* 18.1 (1989), págs. 186-208. DOI: 10.1137/0218012. URL: https://doi.org/10.1137/0218012.

[10] Oded Goldreich, Silvio Micali y Avi Wigderson. «Proofs that Yield Nothing but Their Validity or All Languages in NP Have Zero-Knowledge Proof Systems». En: *Journal of the ACM* 38.3 (1991), págs. 690-728. DOI: 10.1145/116825.116852. URL: https://doi.org/10.1145/116825.116852.

[11] Oded Goldreich. *Foundations of Cryptography. Volume 1: Basic Tools*. Cambridge University Press, 2001. DOI: 10.1017/CBO9780511546891. URL: https://doi.org/10.1017/CBO9780511546891.

[12] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer y Madars Virza. «Zerocash: Decentralized Anonymous Payments from Bitcoin». En: *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, págs. 459-474. DOI: 10.1109/SP.2014.36. URL: https://doi.org/10.1109/SP.2014.36.

[13] Michael Walfish y Andrew J. Blumberg. «Verifying Computations without Reexecuting Them». En: *Communications of the ACM* 58.2 (2015), págs. 74-84. DOI: 10.1145/2641562. URL: https://doi.org/10.1145/2641562.

[14] Gilles Brassard, David Chaum y Claude Crépeau. «Minimum Disclosure Proofs of Knowledge». En: *Journal of Computer and System Sciences* 37.2 (1988), págs. 156-189. DOI: 10.1016/0022-0000(88)90005-0. URL: https://doi.org/10.1016/0022-0000(88)90005-0.

[15] Mihir Bellare y Oded Goldreich. «On Defining Proofs of Knowledge». En: *Advances in Cryptology — CRYPTO '92*. Vol. 740. Lecture Notes in Computer Science. Springer, 1993, págs. 390-420. DOI: 10.1007/3-540-48071-4_28. URL: https://doi.org/10.1007/3-540-48071-4_28.

[16] Manuel Blum, Paul Feldman y Silvio Micali. «Non-Interactive Zero-Knowledge and Its Applications». En: *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing (STOC '88)*. ACM, 1988, págs. 103-112. DOI: 10.1145/62212.62222. URL: https://doi.org/10.1145/62212.62222.

[17] Amos Fiat y Adi Shamir. «How to Prove Yourself: Practical Solutions to Identification and Signature Problems». En: *Advances in Cryptology — CRYPTO '86*. Vol. 263. Lecture Notes in Computer Science. Springer, 1987, págs. 186-194. DOI: 10.1007/3-540-47721-7_12. URL: https://doi.org/10.1007/3-540-47721-7_12.

[18] Mihir Bellare y Phillip Rogaway. «Random Oracles Are Practical: A Paradigm for Designing Efficient Protocols». En: *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS '93)*. ACM, 1993, págs. 62-73. DOI: 10.1145/168588.168596. URL: https://doi.org/10.1145/168588.168596.

[19] David Bernhard, Olivier Pereira y Bogdan Warinschi. «How Not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios». En: *Advances in Cryptology — ASIACRYPT 2012*. Vol. 7658. Lecture Notes in Computer Science. Springer, 2012, págs. 626-643. DOI: 10.1007/978-3-642-34961-4_38. URL: https://doi.org/10.1007/978-3-642-34961-4_38.

[20] Bryan Parno, Jon Howell, Craig Gentry y Mariana Raykova. «Pinocchio: Nearly Practical Verifiable Computation». En: *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, págs. 238-252. DOI: 10.1109/SP.2013.47. URL: https://doi.org/10.1109/SP.2013.47.

[21] Stefanos Chaliasos, Jens Ernstberger, David Theodore, David Wong, Mohammad Jahanara y Benjamin Livshits. *SoK: What Don't We Know? Understanding Security Vulnerabilities in SNARKs*. arXiv:2402.15293 [cs.CR]. 2024. URL: https://arxiv.org/abs/2402.15293.

[22] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer y Madars Virza. «SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge». En: *Advances in Cryptology — CRYPTO 2013*. Vol. 8043. Lecture Notes in Computer Science. Springer, 2013, págs. 90-108. DOI: 10.1007/978-3-642-40084-1_6. URL: https://doi.org/10.1007/978-3-642-40084-1_6.

[23] Jacob T. Schwartz. «Fast Probabilistic Algorithms for Verification of Polynomial Identities». En: *Journal of the ACM* 27.4 (1980), págs. 701-717. DOI: 10.1145/322217.322225. URL: https://doi.org/10.1145/322217.322225.

[24] Richard Zippel. «Probabilistic Algorithms for Sparse Polynomials». En: *Symbolic and Algebraic Computation (EUROSAM '79)*. Vol. 72. Lecture Notes in Computer Science. Springer, 1979, págs. 216-226. DOI: 10.1007/3-540-09519-5_73. URL: https://doi.org/10.1007/3-540-09519-5_73.

[25] Aniket Kate, Gregory M. Zaverucha y Ian Goldberg. «Constant-Size Commitments to Polynomials and Their Applications». En: *Advances in Cryptology — ASIACRYPT 2010*. Vol. 6477. Lecture Notes in Computer Science. Springer, 2010, págs. 177-194. DOI: 10.1007/978-3-642-17373-8_11. URL: https://doi.org/10.1007/978-3-642-17373-8_11.

[26] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh y Michael Riabzev. «Fast Reed-Solomon Interactive Oracle Proofs of Proximity». En: *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Vol. 107. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, 14:1-14:17. DOI: 10.4230/LIPIcs.ICALP.2018.14. URL: https://doi.org/10.4230/LIPIcs.ICALP.2018.14.

[27] Nir Bitansky, Ran Canetti, Alessandro Chiesa y Eran Tromer. «From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again». En: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS '12)*. ACM, 2012, págs. 326-349. DOI: 10.1145/2090236.2090263. URL: https://doi.org/10.1145/2090236.2090263.

[28] Jens Groth. «On the Size of Pairing-Based Non-interactive Arguments». En: *Advances in Cryptology — EUROCRYPT 2016*. Vol. 9666. Lecture Notes in Computer Science. Springer, 2016, págs. 305-326. DOI: 10.1007/978-3-662-49896-5_11. URL: https://doi.org/10.1007/978-3-662-49896-5_11.

[29] iden3. *snarkjs: zkSNARK Implementation in JavaScript and WASM*. Repositorio de código fuente. 2018. URL: https://github.com/iden3/snarkjs.

[30] Ariel Gabizon, Zachary J. Williamson y Oana Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive, Paper 2019/953. 2019. URL: https://eprint.iacr.org/2019/953.

[31] Aztec Labs. *Barretenberg Documentation*. Documentación oficial del backend de pruebas Barretenberg. 2025. URL: https://barretenberg.aztec.network/docs/.

[32] Sean Bowe, Ariel Gabizon y Ian Miers. *Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model*. Cryptology ePrint Archive, Paper 2017/1050. 2017. URL: https://eprint.iacr.org/2017/1050.

[33] Christian Reitwiessner. *EIP-196: Precompiled Contracts for Addition and Scalar Multiplication on the Elliptic Curve alt_bn128*. Ethereum Improvement Proposals. 2017. URL: https://eips.ethereum.org/EIPS/eip-196.

[34] Vitalik Buterin y Christian Reitwiessner. *EIP-197: Precompiled Contracts for Optimal Ate Pairing Check on the Elliptic Curve alt_bn128*. Ethereum Improvement Proposals. 2017. URL: https://eips.ethereum.org/EIPS/eip-197.

[35] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh y Michael Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. Cryptology ePrint Archive, Paper 2018/046. 2018. URL: https://eprint.iacr.org/2018/046.

[36] Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh y Dawn Song. «zkBridge: Trustless Cross-chain Bridges Made Practical». En: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. ACM, 2022, págs. 3003-3017. DOI: 10.1145/3548606.3560652. URL: https://doi.org/10.1145/3548606.3560652.

[37] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio y Jordi Baylina. «Circom: A Circuit Description Language for Building Zero-Knowledge Applications». En: *IEEE Transactions on Dependable and Secure Computing* 20.6 (2023), págs. 4733-4751. DOI: 10.1109/TDSC.2022.3232813. URL: https://doi.org/10.1109/TDSC.2022.3232813.

[38] iden3. *Circom 2 Documentation*. Documentación oficial del compilador circom, versión 2.2. 2025. URL: https://docs.circom.io/.

[39] iden3. *circomlib: Library of Basic Circuits for Circom*. Repositorio de código fuente. 2018. URL: https://github.com/iden3/circomlib.

[40] iden3. *Circom Release Notes*. Archivo RELEASES.md del repositorio oficial del compilador circom. 2025. URL: https://github.com/iden3/circom/blob/master/RELEASES.md.

[41] Shankara Pailoor, Yanju Chen, Franklyn Wang, Clara Rodríguez, Jacob Van Geffen, Jason Morton, Michael Chu, Brian Gu, Yu Feng e Işıl Dillig. «Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs». En: *Proceedings of the ACM on Programming Languages* 7.PLDI (2023), págs. 1510-1532. DOI: 10.1145/3591282. URL: https://doi.org/10.1145/3591282.

[42] Hongbo Wen, Jon Stephens, Yanju Chen, Kostas Ferles, Shankara Pailoor, Kyle Charbonnet, Işıl Dillig y Yu Feng. «Practical Security Analysis of Zero-Knowledge Proof Circuits». En: *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024, págs. 1471-1487. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/wen.

[43] Gavin Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Inf. téc. Yellow Paper. Ethereum Foundation, 2014. URL: https://ethereum.github.io/yellowpaper/paper.pdf.

[44] Nick Johnson. *EIP-658: Embedding Transaction Status Code in Receipts*. Ethereum Improvement Proposals. 2017. URL: https://eips.ethereum.org/EIPS/eip-658.

[45] Ethereum Foundation. *Altair Light Client – Sync Protocol*. Especificaciones de consenso de Ethereum (consensus-specs). 2021. URL: https://github.com/ethereum/consensus-specs/blob/master/specs/altair/light-client/sync-protocol.md.

[46] Ethereum Foundation. *Capella Light Client – Sync Protocol*. Especificaciones de consenso de Ethereum (consensus-specs). 2023. URL: https://github.com/ethereum/consensus-specs/blob/master/specs/capella/light-client/sync-protocol.md.

[47] 0xPARC ZK-ID Working Group. *circom-pairing: Proof-of-Concept Implementations of BLS12-381 Pairings in Circom*. Repositorio de código fuente. 2022. URL: https://github.com/yi-sun/circom-pairing.

[48] noir-lang. *noir: The Noir Programming Language, etiqueta v1.0.0-beta.22*. Repositorio oficial, incluye la documentación de ACIR, ACVM y Brillig en acvm-repo. Consultado el 10 de octubre de 2026. 2026. URL: https://github.com/noir-lang/noir/tree/v1.0.0-beta.22.

[49] noir-lang. *Noir Documentation, versión 1.0.0-beta.22*. Fuente de la documentación en la etiqueta v1.0.0-beta.22. Consultada el 10 de octubre de 2026. 2026. URL: https://github.com/noir-lang/noir/tree/v1.0.0-beta.22/docs/docs.

[50] noir-lang. *@noir-lang/noir_js, versión 1.0.0-beta.22*. Paquete npm. 2026. URL: https://www.npmjs.com/package/@noir-lang/noir_js/v/1.0.0-beta.22.

[51] Aztec Labs. *Barretenberg, etiqueta v5.0.0-nightly.20260522*. Código fuente y documentación en el repositorio aztec-packages. Consultado el 10 de octubre de 2026. 2026. URL: https://github.com/AztecProtocol/aztec-packages/tree/v5.0.0-nightly.20260522/barretenberg.

[52] Aztec Labs. *@aztec/bb.js, versión 5.0.0-nightly.20260522*. Paquete npm, incluye el binario bb de la misma versión. 2026. URL: https://www.npmjs.com/package/@aztec/bb.js/v/5.0.0-nightly.20260522.

[53] Ariel Gabizon y Zachary J. Williamson. *plookup: A Simplified Polynomial Protocol for Lookup Tables*. Cryptology ePrint Archive, Paper 2020/315. 2020. URL: https://eprint.iacr.org/2020/315.

[54] Ulrich Haböck. *Multivariate Lookups Based on Logarithmic Derivatives*. Cryptology ePrint Archive, Paper 2022/1530. 2022. URL: https://eprint.iacr.org/2022/1530.

[55] Aztec Protocol. *Ignition Verification: Repository to Verify Contributions to the AZTEC Ignition Ceremony*. Repositorio de código fuente. 2020. URL: https://github.com/AztecProtocol/ignition-verification.

[56] Succinct Labs. *SP1: repositorio oficial, etiqueta v6.2.2*. Incluye la biblioteca SLOP y los crates del ejecutor, del sistema de pruebas Hypercube, del SDK y de la zkVM. Consultado el 10 de octubre de 2026. 2026. URL: https://github.com/succinctlabs/sp1/tree/v6.2.2.

[57] Succinct Labs. *SP1 Documentation (SP1 V6)*. Documentación en línea; las páginas consultadas no indican una versión de parche. Consultada el 10 de octubre de 2026. 2026. URL: https://docs.succinct.xyz/docs/sp1/introduction.

[58] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, versión 20240411*. Especificación oficial. 2024. URL: https://github.com/riscv/riscv-isa-manual/releases/tag/20240411.

[59] Succinct Labs. *sp1-contracts: contratos verificadores de SP1, etiqueta v6.1.0*. Repositorio de código fuente. 2026. URL: https://github.com/succinctlabs/sp1-contracts/tree/v6.1.0.

[60] Succinct Labs. *SP1 Hypercube Security Model*. Documentación de SP1 V6, sección de seguridad. Consultada el 11 de octubre de 2026. 2026. URL: https://docs.succinct.xyz/docs/sp1/security/security-model.

[61] Shahar Papini y Ulrich Haböck. *Improving Logarithmic Derivative Lookups Using GKR*. Cryptology ePrint Archive, Paper 2023/1284. 2023. URL: https://eprint.iacr.org/2023/1284.

[62] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh y Ron D. Rothblum. *Jagged Polynomial Commitments (or: How to Stack Multilinears)*. Cryptology ePrint Archive, Paper 2025/917. Revisión menor de la publicación en EUROCRYPT 2026. 2025. URL: https://eprint.iacr.org/2025/917.

[63] Hadas Zeilberger, Binyi Chen y Ben Fisch. *BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes*. Cryptology ePrint Archive, Paper 2023/1705. Revisión menor de la publicación en CRYPTO 2024. 2023. URL: https://eprint.iacr.org/2023/1705.

[64] Consensys. *gnark: High-Performance zk-SNARKs in Go*. Repositorio de código fuente. 2026. URL: https://github.com/Consensys/gnark.

APÉNDICE

A.1. A